

UPPSALA
UNIVERSITET

Commutator Analysis for Game Design

A quantitative method for structural game coupling analysis

Department of Game Design

Author: Zhenghao Wang

Master Thesis in Game Design, 30 hp

Supervisor: Eliel Camargo-Molina

Examiner: Magnus Johansson

Month, Year: June, 2026

Abstract

This thesis introduces a novel, quantitative method for structural game coupling analysis by adopting the algebraic commutator ($[A,B] = AB - BA$) from physics. While traditional game analysis frameworks (such as MDA and Machinations) focus primarily on specific scenario outcomes, they lack formal algebraic tools to diagnose the structural origins of action coupling. To address this gap, this study establishes a theoretical framework that maps linear, state-independent game actions into numerical and symbolic matrices, transforming commutators into an analytical language for designers. Non-zero elements within the commutator matrix explicitly reveal the location, strength, direction, and source of parameter couplings. Through a case study of the card game prototype *Chaos Fighter*, this study systematically demonstrates how pairwise, combinatorial, and nested commutators distinguish different coupling structures and uncover hidden interactions. Furthermore, global coupling strength and frequency heatmaps are utilized to identify systemic design imbalances, hot spots, and strategic blind spots. Ultimately, this study provides a highly efficient, objective structural diagnosis tool that complements traditional playtesting and enhances mathematical rigor in game design.

Contents

1 Introduction 5
1.1 Interaction and Coupling 5
1.2 An Inspiration from Physics 5
1.3 Research Question 6
1.4 Contributions 6
1.5 Scope and Limitations 7

2 Background and Related Work 8
2.1 Coupling in Game Design 8
2.2 Existing Formal Methods for Game Analysis 8
2.3 State Space 9

3 Theoretical Framework 11
3.1 Definition of Coupling 11
3.2 Design Task 11
3.3 Game Action Matrix Representation 12
3.4 Algebraic Commutator 14
3.4.1 Comparison with Traditional Methods 16
3.4.2 Symbolization 17
3.4.3 Logical Inverse vs Mathematical Inverse 17
3.4.4 Decomposition of Action Combinations 19
3.5 Generalization 21
3.6 Declaration: Scope of Linearity 21

4 Case Study - Chaos Fighter 22
4.1 Game Introduction 22
4.1.1 Design Motivation 22
4.1.2 Game Rules 22
4.2 Game State and Card Pool 23
4.2.1 Game State Representation 23
4.2.2 Card Pool 23
4.2.3 Card Matrix Representation 24
4.3 Commutator Analysis for Existing Cards 25
4.3.1 Combination Training - Swords Made from Armor 25
4.3.2 Swords Made from Armor - Stealing the Future 26
4.3.3 Combination Training - Stealing the Future 26
4.4 Commutator Analysis for Adding a New Card 27
4.4.1 Single - Single Commutator 27
4.4.2 Single - Combined Commutator 27
4.4.3 Nested Commutator 30
4.4.4 Summary 30
4.5 Statistical Commutator Analysis 31
4.5.1 Coupling Strength Heatmap 32
4.5.2 Coupling Frequency Heatmap 33
4.6 Summary of Case Study 34

5 Discussion 36
5.1 Formalization and Mathematics 36
5.1.1 The Commutator as a Metric for Structural Coupling 36

5.1.2 From Analysis to Design ... 36
5.1.3 Translation Process ... 36
5.2 Game Structures and Applicable Scope ... 36
5.2.1 The Challenges of Tic-Tac-Toe ... 36
5.2.2 Logical Structures and Dynamical Structures ... 37
5.2.3 Applicable Scope of Commutator Analysis ... 37
5.3 Atomic White-Box Assumption and Games as Gray-Boxes ... 38
5.3.1 Atomic White-Box Assumption ... 38
5.3.2 Games as Gray-Boxes ... 38
5.4 Commutator and Emergence ... 39
5.4.1 Emergence and the Black-Box ... 39
5.4.2 The Relationship Between Commutator and Emergence in Games ... 39
5.4.3 Designed Emergence? ... 39
5.4.4 The Role and Position of Commutator Analysis ... 40
5.5 Limitations and Boundaries ... 40

6 Conclusions and Future Work ... 41
6.1 Addressing the Research Questions ... 41
6.2 Theoretical Contributions ... 41
6.3 Contributions of the Case Study ... 41
6.4 Outlook for Nonlinear Scenarios ... 41
6.5 More General Commutators ... 42

Appendix A: Value of Commutators in Case Study ... 44

Appendix B: Mathematica Source Code ... 47

1. Introduction

1.1 Interaction and Coupling

The core characteristic of games lies in their interactivity, which is the key factor that distinguishes them from other traditional passive media, such as books, images, and films(Aarseth, 1997; Juul, 2011). Generally speaking, interactivity depends on the player's actions. When players take actions, the game changes, and interactivity is realized(Tekinbas & Zimmerman, 2003). Therefore, player actions are the concrete vehicle for a game's interactivity and constitute the focus of our game research. Among the many attributes of actions, one is particularly critical yet often treated implicitly: how actions influence one another—that is, the coupling of actions. This coupling forms complex dynamic interactions within game systems(Hunicke et al., 2004) and often serves as a crucial component of the core gameplay experience (such as emergent gameplay)(Adams & Dormans, 2012).

Consider the following scenario: In a role-playing game, a player is engaged in combat with an enemy. Kinetic Overload: Increases the player's next skill damage by 30%. Mech Synergy: Inflicts both the player's base damage and the mech's base damage simultaneously. If the player wants to maximize damage, they must carefully plan the order in which skills are cast. It is clear that by using Kinetic Overload first, followed by Mech Synergy, the player can achieve higher damage output.

In this scenario, we find that the key to victory lies in the order in which skills are cast. This sequence dependency in gameplay outcomes is, in fact, a direct manifestation of coupling—where one action influences another. This exists in nearly all games, such as numerical card games. We need to consider aspects like deck construction and play order. And in the vast majority of cases, these considerations are predicated on the coupling relationships between cards. When building a deck, we consider the synergy between cards, which is essentially a form of coupling. During actual gameplay, analyzing the order of play also involves analyzing the coupling between the cards about to be played. Similarly, in more general game genres, coupling is also a crucial component. For example, combos in fighting games, puzzles in adventure games, and the order of actions in strategy games, and so on.

Currently, our analysis and understanding of these game designs and the coupling relationships behind them rely primarily on methods such as test-iteration cycle, designers' intuition, or numerical simulations. It is not difficult to see the benefit of Kinetic Overload to Mech Synergy, and most players participating in game testing can also understand this relationship. The key point here is that the above analytical methods examine the outcomes of coupling in specific scenarios, rather than the coupling itself. Therefore, for game actions—a fundamental and important phenomenon—the coupling of operations within the game still lacks a direct, formalized research method.

1.2 An Inspiration from Physics

In other academic disciplines, there has been extensive and in-depth research on the phenomenon of coupling. For example, in information theory, mutual information is used to measure the amount of information shared between two variables(Cover, 1999); in computer science and network science, explicit connection relationships among elements in a system are typically quantified directly by adjacency matrices(Newman, 2018); and in control theory, relative gain arrays (RGAs) are widely used to assess the degree of interaction between loops in multi-input multi-output systems(Skogestad & Postlethwaite, 2005). These methods can quantitatively analyze the

strength of coupling, but they are primarily exploratory in nature—that is, they analyze observed coupling effects. They can illustrate how strong the coupling effects are, but they cannot account for the structural origins of the coupling.

In physics, the commutator $[A,B] = AB - BA$ provides a relatively deeper, structural analysis of coupling. In this expression, A and B are operators, conceptually analogous to actions in a game. The operators A and B themselves serve directly as mathematical objects; the commutator reveals the presence or absence of coupling and its effects through algebraic operations on the operators themselves. Take the famous uncertainty principle as an example: $[x, p] = i\hbar$. This is derived directly from the algebraic structure of the position and momentum operators, without relying on any specific particle(Von Neumann, 2018). At the same time, the result $i\hbar$ transcends the characterization of the coupling strength between two physical quantities and is universally present in the formulations of many physical laws. The commutator reveals the intrinsic, inherent coupling relationship between operators. This goes a step further than exploratory methods.

Therefore, compared to methods from other disciplines, the commutator is better suited for artificially constructed structures such as games. Games are, by nature, artificially designed systems, with actions clearly defined by the designer. We possess complete definitions of operations, but lack the tools to analyze the coupling relationships between them. The commutator not only has the potential to fill this gap but is also highly intuitive for games. It can be directly understood as the net difference between two sequences of actions, which is directly relevant to game design and testing. Furthermore, the matrix representation of the commutator is naturally suited to data processing and analytical methods used in classical computing. If game operations are matrixified using appropriate methods, we can directly employ efficient hardware and software tools for systematic analysis.

1.3 Research Question

Finally, we pose the core research question of this paper:

Can commutation relations and commutators serve as a direct, structural, quantitative measure of game action coupling, revealing structural information that is difficult or impossible to obtain through existing testing and simulation methods? If so, how can they be applied in practical game design?

1.4 Contributions

This paper makes the following three contributions:

First, we establish a theoretical framework for algebraic commutators as a tool for measuring game coupling. We introduce matrix algebraic operations and commutation relations into the context of game design. By interpreting rules through matrix elements (rows = goals, columns = sources), symbolic generalization, and distinguishing between logical inverses and mathematical inverses, we transform algebraic commutators from abstract mathematical concepts into an analytical language that designers can operate with.

Second, through a specifically constructed card game prototype, Chaos Fighter, we systematically demonstrate the multi-level applications of algebraic commutator analysis. In the case study, by representing matrices through partitioning, we further clarified the typological classifications of couplings between different parameters; combinatorial commutators revealed 'internal order sensitivity within combinations' that traditional testing cannot reach, along with its hierarchical differentiation. Heatmaps of coupling strength and frequency provided a global structural diagnosis, identifying coupling hotspots, blind spots, and pivotal parameters.

Third, we systematically compared commutator analysis with traditional design methods based on testing and intuition. On both the dimensions of the content of coupling information revealed and the efficiency of its acquisition, we demonstrated that commutator analysis can provide structural coupling information that traditional methods cannot directly obtain, including the precise

location, strength, direction, and source of coupling parameters, as well as the global distribution of coupling across the card pool.

1.5 Scope and Limitations

The analytical framework in this paper focuses on linear, state-independent representations of game actions—that is, each action is described by a fixed matrix, and its behavior does not change with the current game state. However, state-dependent designs (e.g., conditional triggers, threshold effects) and randomness mechanisms are widespread in actual games. Such context-sensitive actions require analysis under more complex mathematical representations, which will serve as a direction for future research.

Furthermore, the current framework is analytical rather than generative: it is used to evaluate and reveal coupling structures in existing game designs, rather than to automatically generate or recommend new design solutions. This limitation also clarifies the scope of this research: to provide formal analytical tools for game design, rather than to replace the design process itself.

2. Background and Related Work

2.1 Coupling in Game Design

The formal analysis of game design began with the MDA framework, which divides games into three layers: Mechanics, Dynamics, and Aesthetics (Hunicke et al., 2004). In the MDA framework, Mechanics refers to the description of game rules at the data and algorithmic levels, while Dynamics refers to the runtime behavior generated when players interact with Mechanics. Coupling—where the effect of one action varies depending on the order or presence of another action—lies precisely at the intersection of mechanics and dynamics. Its existence is defined by mechanics, but its effects manifest at the dynamic level. However, while the MDA framework provides a layered perspective for understanding games, it does not offer direct analytical tools for this ubiquitous and important phenomenon of coupling.

Building upon the MDA framework, the Layered Tetrad further expands game analysis into a three-tiered structure of "Inscribed Layer, Dynamic Layer, Cultural Layer," and decomposes games into a tetrad of "Mechanics, Aesthetics, Narrative, Technology" (Bond, 2014). The analytical focus of this framework lies in the emergent relationships between layers: how rule-setting in the Inscribed Layer generates behavior in the Dynamic Layer, and how this, in turn, forms social meaning in the Cultural Layer. However, similar to the MDA framework, the Layered Tetrad focuses on causal connections between layers rather than structural features within a single layer. Action coupling, as an inherent property of the mechanics, lacks a corresponding conceptual position within the analytical language of the Layered Tetrad.

Salen and Zimmerman's Rules of Play: Game Design Fundamentals is a seminal theoretical work in the field of game design (Tekinbas & Zimmerman, 2003). Centered on the concept of "meaningful play," the book defines a game as "a system in which players engage in rule-defined virtual conflicts that produce quantifiable outcomes." This classic "rules-play-culture" triadic framework has profoundly influenced subsequent game research paradigms. However, Rules of Play focuses on how rules "generate" meaningful behavior, rather than how the actions defined by rules "interact" with one another. In discussing the classification of operational, constitutive, and implicit rules, the authors focus on the comprehensibility and executability of rules, rather than the structural relationships between them. The coupling of potential sequential dependencies between actions is not abstracted as an independent design dimension.

2.2 Existing Formal Methods for Game Analysis

Several important methods have been proposed for the formal analysis of game mechanics and dynamics.

The Machinations framework is one of the most representative methods among them (Dormans, 2012). It uses a formal language based on resource flow graphs to express the structure of a game's internal economic system, allowing designers to construct resource nodes and transformation relationships through a graphical interface and observe the system's dynamic behavior via a built-in simulation engine. Machinations can effectively model resource flows and system feedback, making it highly practical for game balance analysis. Its core lies in simulating the outcomes of resource flows, but it lacks an algebraic description of the commutativity or order sensitivity of operations. Designers can use Machinations to observe that "changing the order of operations leads to different resource distributions," but they cannot structurally derive precise information such as "which pairs of operations are coupled, between which two attributes the coupling exists, or which parameters determine the strength of the coupling."

Formal game grammar offers an alternative formalization approach. Drawing on frameworks from formal languages and automaton theory, this method represents game rules as a system of production rules or a state transition diagram, using strict syntax to describe the relationship between player actions and game state transitions (Nelson & Mateas, 2007). Formal grammar can precisely describe how rules combine and supports theoretical analysis of properties such as rule equivalence and game complexity. The focus of formal grammar lies on "the validity of rules" and "state transition relationships," rather than "the differences in outcomes resulting from the permutation of action sequences." It can express that "a player may execute A before executing B," but cannot directly quantify the origin or magnitude of the difference in outcomes between "A first, then B" and "B first, then A."

Combinatorial game theory provides a rigorously algebraic analytical framework. This theory applies to specific subclasses of two-player games of perfect information. It maps game states to equivalence classes using algebraic tools such as Sprague-Grundy values, and analyzes the game's strategic structure based on this mapping (Berlekamp et al., 2004). Combinatorial game theory is mathematically elegant, but its scope of application is highly limited: it requires games to have specific structures (such as last-move advantage) and focuses primarily on win-loss outcomes rather than the numerical changes and combinatorial effects commonly found in resource management. For the vast majority of strategy games centered on resource flows and attribute combinations, the tools of combinatorial game theory are difficult to directly apply.

In summary, existing formal methods share a fundamental limitation when analyzing the phenomenon of coupling: they analyze the outcomes of coupling under specific states rather than the structure of coupling itself. While designers can use these methods to detect the presence of coupling, they cannot obtain structural information regarding the precise location, strength, source of parameters, or global distribution of coupling channels. It is precisely this gap that this study aims to fill through algebraic commutator analysis.

2.3 State Space

Commutator analysis can essentially be classified as a state space analysis method. In game studies and many other fields, the state space is a ubiquitous and well-established concept.

In the fields of computer science and artificial intelligence (AI), the state space is viewed as a discrete, graph-theoretic mathematical model, commonly used for formally solving complex problems and automated planning. Here, the state space is a set comprising all possible states, with each state representing the specific physical or logical configuration of the problem at a given moment. The actions of players or agents are abstracted as operators, which connect different states as directed edges and drive transitions between states (Russell, 2010). The core task of AI is to find an optimal path from the initial state to the goal state within this vast graph structure using state-space search algorithms. When a system involves multiple variables or is sensitive to the order of operations, the number of states grows exponentially with path depth, leading to the well-known "state-space explosion" problem. Consequently, research in this field focuses on how to efficiently navigate and simplify the state space through methods such as partial order reduction and heuristic pruning (Baier & Katoen, 2008).

In modern control theory, state-space representation is a powerful tool for transforming complex physical or engineering systems into linear algebraic matrix equations in continuous or discrete time. Unlike the discrete symbolic states in artificial intelligence, the state space in control theory is a multidimensional continuous geometric space formed by a set of state variables as basis vectors, and the instantaneous behavior of the system is uniquely determined by a state vector in this space (Brogan, 1985). The system's evolution is encoded in state equations, where the system matrix describes the intrinsic coupling structure among the system's internal components, while the input matrix determines how external control actions are mapped onto and influence the state space. From a control theory perspective, the state space focuses not only on the results of state transitions but also on the system's trajectory evolution, asymptotic stability, controllability,

and observability within the space. When dealing with complex nonlinear systems, control theory also introduces tools from differential geometry, treating the state space as a manifold and utilizing algebraic tools such as Lie brackets to measure the instantaneous coupling effects triggered by continuous action flows within the space (Khalil & Grizzle, 2002).

In ludology and game design theory, the state space serves as both the mathematical essence of the game's rule system and the stage for the player's agency. Jesper Juul said: "From a formal perspective, a game is a state machine. It is a machine that has an initial state, accepts inputs (actions) from players, and changes its state according to a set of rules" (Juul, 2011). The game's initial interface, item values, and level progress collectively constitute the current state, while every action the player performs on the controller serves as an input command that triggers changes within the state space.

3. Theoretical Framework

3.1 Definition of Coupling

Recall the example from the introduction, such as the effects of a buff skill and an attack skill on a single enemy. When facing an enemy (a single initial state s), the buff skill clearly influences the outcome of the attack skill. Casting the buff skill before the attack skill yields a different final enemy health value than casting them in the reverse order. This is an example of coupling for a single state.

Therefore, we can attempt to provide a definition of operational coupling in a game.

Operations A and B are coupled with respect to a single state if $A(B(s)) \neq B(A(s))$

This means that one operation influences the outcome of another; it can also be intuitively understood as the fact that swapping the order of the operations yields different results.

In most cases, however, we are not studying just a single game segment, but rather a series of consecutive, specific game scenarios. A typical scenario is when the game is in a specific weather condition that affects the outcome of the player's actions—for example, using lightning-based magic in the rain will cause self-damage. Throughout the entire gameplay process during rain, the game's state can be classified as belonging to the set of states "in rainy weather." Designers frequently need to analyze the characteristics of game actions within specific contexts, constraints, and other local scopes. This implies that we need to focus more on analyzing the coupling relationships of game actions within specific sets of states. Therefore, a more common scenario involves analyzing and studying the coupling relationship between two operations within a given set of states—that is, within a specific context.

Operations A and B are coupled with respect to a set of states if, for a set of states S, there exists an s_0 such that $A(B(s_0)) \neq B(A(s_0))$

This means that as long as there exists a single state for which the operations are coupled, the two operations are considered coupled within that local scope. It is important to note that we are using an existence-based definition rather than a universality-based definition. A possible, opposite definition would be that two operations are coupled with respect to a set of states only if they are coupled for every state in that set. There are several reasons why we do not adopt this definition. 1. In game design contexts, it is counterintuitive to consider two operations uncoupled for a set simply because there exists a single uncoupled state. Furthermore, ignoring other coupled states would drastically reduce the validity of the analysis, making its applicability extremely limited. 2. Mathematical applicability: When adapting to concepts such as linear algebra and group theory in subsequent sections, a universality-based definition cannot align with mathematical structures, rendering mathematical tools ineffective.

In the introduction, inspired by quantum mechanics, we derived a basic commutator form: $[A,B] = AB - BA$. However, for the commutator to serve as an analytical method for game design, its value must be validated and expanded upon within specific design contexts. Therefore, starting from a specific design task, we will demonstrate step-by-step how the commutator analyzes games and the design insights it yields.

3.2 Design Task

To more clearly demonstrate the significance of coupling effects for games, we have constructed a game scenario centered on the circulation of resources: players exchange elemental energy with

the environment. In this scenario, as a designer, we face the following task: to construct interactions between the character and the environment centered on Aero and Water Energy. Consequently, certain attributes of the game can be represented by four parameters: player Aero Energy, player Water Energy, environmental Aero Energy, and environmental Water Energy. We can list these four parameters side by side in the following form:

$$s = \begin{bmatrix} p_a \\ p_w \\ e_a \\ e_w \end{bmatrix}$$

s represents the game state, which is described by the four parameters we have defined. This can be referred to as the "state vector." A preliminary concept is to design a character named "Feng Shui Master," who can manipulate both their own and the environment's Feng Shui energy through skills. We need to design skills that allow the character to influence these four parameters while remaining consistent with the game's lore.

3.3 Game Action Matrix Representation

We start with the simplest form and present the following two skill design drafts:

Aero-Driven-Water(AW): Uses a small air current to project Water Energy outward (projecting a small amount of Aero Energy and a large amount of Water Energy into the environment)

Water-Induced-Aero(EWA): Consumes Water Energy to increase the density of external Aero Energy (projecting a small amount of Water Energy into the environment to boost the environmental Aero Energy)

These are single-use skills. We now provide specific parameter descriptions:

Aero-Driven-Water: The player transfers 20% of their current Aero Energy and 50% of their Water Energy into the environment

Water-Induced-Aero: The player transfers 30% of their Water Energy into the environment, increasing the environmental Aero Energy by 40%.

We can express the effects of these two skills using the previous parameter set:

$$AW \cdot s = \begin{bmatrix} 0.8p_a \\ 0.5p_w \\ e_a + 0.2p_a \\ e_w + 0.5p_w \end{bmatrix}, \qquad EWA \cdot s = \begin{bmatrix} p_a \\ 0.7p_w \\ 1.4e_a \\ e_w + 0.3p_w \end{bmatrix}$$

$AW \cdot s$ represents the new state of the game after casting Aero-Driven-Water: the player has 80% Aero Energy remaining and 50% Water Energy remaining , which have been transferred to the environment.

The results of each parameter's individual operation can be calculated separately:

$$\begin{aligned} pa' &= (1-0.2)pa \\ pw' &= (1-0.5)pw \\ ea' &= ea + 0.2pa \\ ew' &= ew + 0.5pw \end{aligned}$$

The individual calculations for each parameter are straightforward, but we need four expressions. Can we combine them? In other words, can the operations be expressed as a set of parameters, just like the game state? Matrix operations were created for this very purpose; all of the above operations can be written as a single parameter matrix (see Figure 3.3).

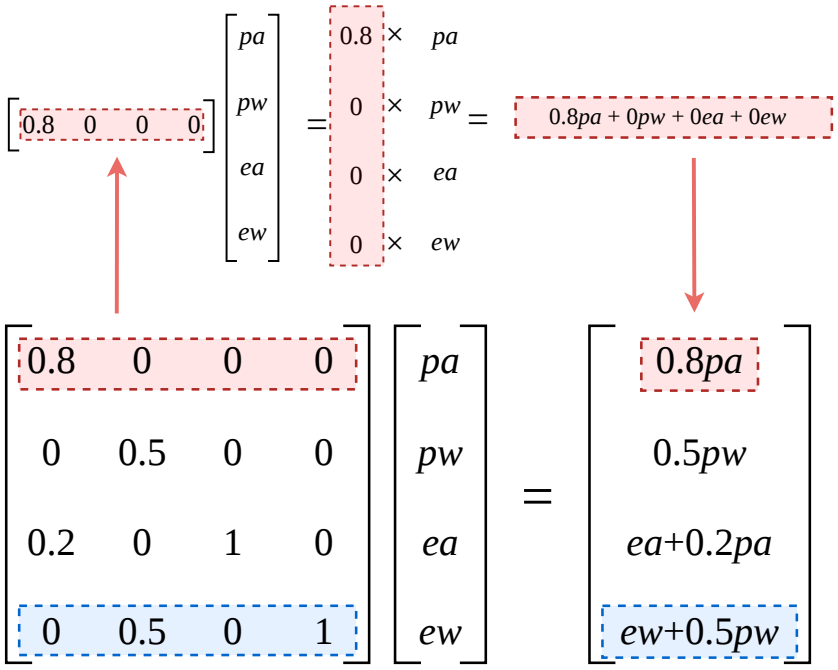


Figure 3.1. Matrix representation of the game action Aero-Driven-Water (*AW*). We have illustrated the rules for matrix operations in the figure. Based on matrix operations and state vectors, we can fully represent all effects of a skill using a single numerical matrix. In this case, each character skill corresponds one-to-one with a numerical matrix.

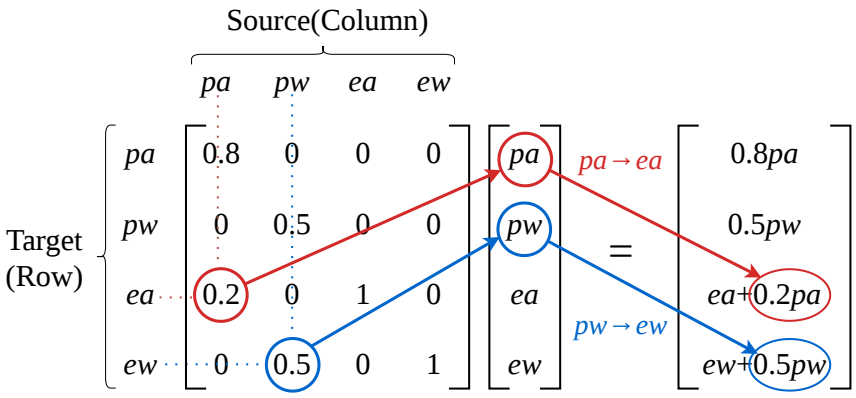


Figure 3.2. Interpretation rules for elements in the game action matrix (*AW*). The rows of the matrix are interpreted as targets. This means that all matrix elements in a given row will influence the parameter corresponding to that row. The columns of the matrix, on the other hand, are interpreted as sources. The column in which a matrix element is located indicates which parameter, in combination with that element, will influence the target. The target-source relationship is established based on the arrangement of the parameters within the state vector itself, as shown in the figure. In the figure, the matrix elements on the diagonal can be interpreted as scaling factors for the parameters themselves, while the off-diagonal elements represent cross-parameter effects. The "0.2" in the first column of the third row indicates that the target parameter 'ea' is influenced by the parameter 'pa', with a coefficient of 0.2.

Similarly, we can compute the matrix representation of EWA, ultimately obtaining:

$$AW = \begin{bmatrix} 0.8 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0.2 & 0 & 1 & 0 \\ 0 & 0.5 & 0 & 1 \end{bmatrix}, \quad EWA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.7 & 0 & 0 \\ 0 & 0 & 1.4 & 0 \\ 0 & 0.3 & 0 & 1 \end{bmatrix}$$

A matrix can represent the results of four operations at once, but for design purposes, how should we interpret these numbers in the matrix? From Figure section 3.3, we can see that the matrix elements actually express the flow relationships between different energies. If we treat the parameter combinations shown in the figure as coordinates, the matrix elements take on specific meanings

Rules for interpreting matrix elements: The element a_{ij} in the i-th row and j-th column represents the "influence of parameter j on parameter i" during a single operation. Rows represent the target (where the influence is applied), and columns represent the source (where the influence comes from).

For example, in the third row and first column of AW, since the value is non-zero, it indicates that *ea* is influenced by *pa*. More specifically, for AW, there is a change in environmental Aero Energy based on the player's Aero Energy. It should also be noted that this influence is unidirectional. The 0.2 in the third row, first column represents the environment gaining 20% of the player's Aero Energy, rather than a "transfer" as described in the skill. Therefore, 0.2 only represents the portion "transferred into environmental Aero Energy," while the player's outflow requires a separate element to describe. This corresponds to the elements on the diagonal, i.e., those in the same row and column. The value 0.8 in the first row and first column means that 80% of the player's Aero Energy is retained, i.e., 20% is transferred out. Inflows and outflows occupy two different positions in the matrix—this is precisely why "transfer" is split into two parts in the matrix representation.

The entire transfer process is actually divided into two parts in the matrix, which is something we need to pay attention to.

Therefore, when we represent an operation as a matrix, we gain more than just the convenience of a single operation. More importantly, by observing the structure of the matrix, we can clearly understand which parameters in this operation influence one another. Diagonal elements represent "self-preservation/scaling," while off-diagonal elements represent "cross-parameter interactions." The effect of any skill (in a linear scenario) can be fully described using these two types of matrix elements.

3.4 Algebraic Commutator

Next, we need to investigate the mutual influence between these two skills—that is, their coupling relationship.

We can start by comparing the differences in results under different orders:

$$AW \cdot EWA \cdot s - EWA \cdot AW \cdot s = \begin{bmatrix} 0 \\ 0 \\ -0.08 p_a \\ 0 \end{bmatrix}$$

This result demonstrates that swapping the order of the operations causes variations in environmental Aero Energy. More generally, the coupling between the two skills affects the environmental Aero Energy under different operation sequences. Next, following the form $[A,B] = AB - BA$, we calculate the commutator $[AW, EWA]$

$$[AW, EWA] = (AW \cdot EWA) - (EWA \cdot AW)$$

$$[AW, EWA] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -0.08 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

We observe that there is a non-zero entry of -0.08 in the first column of the third row.

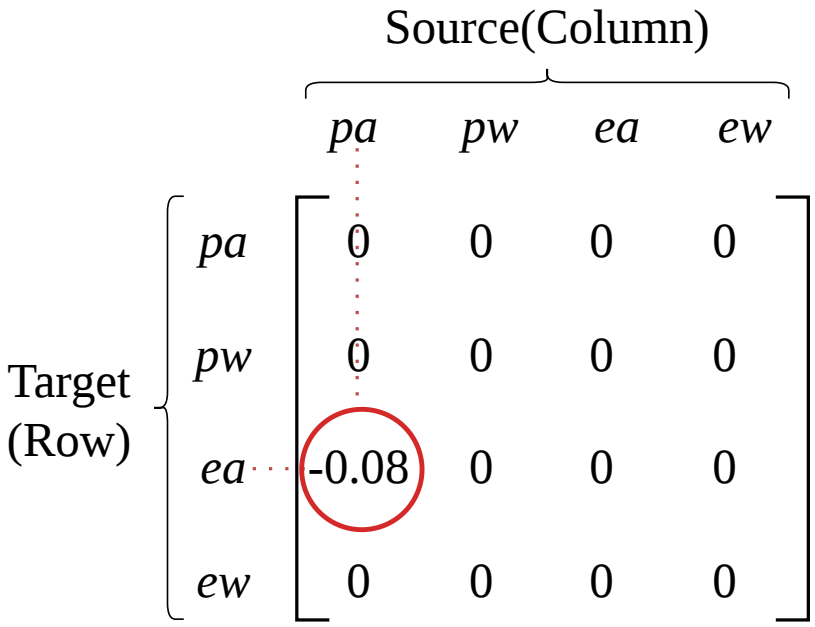


Figure 3.3. Matrix representation of the commutator ($[AW, EWA]$). In the commutator matrix, the target-source relationship remains valid. We further interpret the non-zero matrix elements as coupling channels. The matrix element of -0.08 at the ea-pa node in the figure indicates the presence of a coupling channel between the two operations. This is a coupling effect originating from pa that influences ea. Furthermore, we note that the diagonal entry at the pa-ea position is 0. This indicates that parameter ea does not influence pa in this coupling effect. Therefore, the coupling channel represented by the matrix element in the commutator matrix represents a unidirectional influence relationship. If the parameters were bidirectionally coupled, then both coupling channels—pa-ea and ea-pa—would be nonzero. Furthermore, coupling at the operation level is bidirectional, as the existence of coupling presupposes the presence of two operations.

Earlier, when analyzing individual skills, we established a set of rules for interpreting matrix elements: Rows represent targets, columns represent sources, and a_{ij} denotes "the influence of parameter j on parameter i." However, the commutator matrix does not correspond to a single skill and cannot be executed directly in the game. It requires a corresponding set of interpretation rules.

The commutator matrix retains the same row-column coordinate system—rows still represent targets (parameters affected by the coupling), and columns still represent sources (parameters on which the coupling is based). However, the interpretation must shift from "the effect of an operation" to "the effect of a sequence difference":

Interpretation rule for elements of the commutator matrix: An element $[A,B]_{ij}$ in row i and column j that is non-zero indicates a unidirectional coupling channel from "parameter j to parameter i" between the two operations. This means that different casting orders of A and B will affect the final impact of parameter j on parameter i. The magnitude of this difference is $[A,B]_{ij}$.

Taking our computational results as an example: in $[AW, EWA], (3,1) = -0.08$, which indicates:

Location of coupling: Between environmental Aero Energy (row 3) and player Aero Energy (column 1)

Strength of coupling: Under the two sequences, the contribution of player Aero Energy to environmental Aero Energy differs by $-0.08p_a$

Direction of coupling: The negative sign indicates that when EWA is executed before AW, the increase in environmental Aero Energy is $0.08p_a$ less than in the reverse order.

This explains the discrepancy we initially observed through state comparison: the difference of $-0.08p_a$ in environmental Aero Energy stems precisely from the non-zero term at the commutator position (3,1).

We need to specifically clarify two easily confused concepts: "coupling between operations" and "coupling between parameters within a coupling relationship." The latter differs from the general notion that "coupling implies mutual influence between two entities." In this context of coupling between operations, the parameters themselves are not necessarily bidirectionally coupled. In the current example, $[AW, EWA]$ is non-zero only at (3,1), while (1,3) is zero, indicating unidirectional coupling between the parameters. Player Aero Energy influences the order sensitivity of environmental Aero Energy, but the reverse is not true. If there were bidirectional, mutual coupling between parameters, we would require not only that row i and column j are non-zero, but also that row j and column i are non-zero. That is: in a coupling, there is not only an influence of parameter j on parameter i, but also an influence of parameter i on parameter j. Therefore, commutator can help us more clearly analyze unidirectional and bidirectional couplings, thereby serving potential design objectives. We will elaborate further in subsequent card game examples.

3.4.1 Comparison with Traditional Methods

When comparing the interpretation of the commutator calculation results with those obtained through possible traditional game testing:

1. The casting order of the two skills affects the environmental Aero Energy, and this effect depends solely on the player's initial Aero Energy.

2. The other three parameters are not affected by the skill casting order.

3. If EWA is executed before AW, the final environmental Aero Energy will be lower than when the order is reversed.

The most significant advantage of interpreting the commutator calculations is that we can arrive at similar conclusions through computation without the need for testing, and the results are more quantitative. However, it should also be noted that commutator calculations cannot completely replace traditional testing. This is because traditional testing encompasses additional dimensions, such as gameplay experience and so on.

Furthermore, through traditional numerical calculations, we can also verify the conclusions provided by the commutator.

The impact of $EWA - AW$ on environmental Aero Energy:

$$e_a^{(1_1)} = e_a \cdot (1 + 0.4) = 1.4e_a \tag{3.1}$$

$$e_a^{(1_2)} = e_a^{(1_1)} + p_a^{(1_1)} \cdot 0.2 = 1.4e_a + 0.2p_a \tag{3.2}$$

The effect of $AW - EWA$ on environmental Aero Energy:

$$e_a^{(2_1)} = e_a + p_a \cdot 0.2 \tag{3.3}$$

$$e_a^{(2_2)} = e_a^{(2_1)} \cdot 1.4 = (e_a + 0.2p_a) \cdot 1.4 = 1.4e_a + 0.28p_a \tag{3.4}$$

Final difference:

$$\Delta e_a = (1.4e_a + 0.2p_a) - (1.4e_a + 0.28p_a) = -0.08p_a$$

The results of the traditional numerical calculations for the remaining parts are also 0, consistent with the results of the commutator calculations, and will not be repeated here.

3.4.2 Symbolization

Furthermore, all of our previous calculations were based on specific numerical values, whereas in design practice, it is common to adjust and analyze these values. In fact, another major advantage of matrix operations is the ability to perform symbolic computations without relying on specific parameter expressions. If we replace the fixed numerical parameters in the two skills given earlier with symbolic parameters, where r_a, r_w represent the ratios at which the player transfers their Aero Energy and Water Energy to the environment in the "Aero-Driven-Water" skill, and η_w, g_a represent and the ratio by which the environmental Aero Energy is increased in the "Water-Induced-Aero" skill, we obtain the following form:

$$AW = \begin{bmatrix} 1-r_a & 0 & 0 & 0 \\ 0 & 1-r_w & 0 & 0 \\ r_a & 0 & 1 & 0 \\ 0 & r_w & 0 & 1 \end{bmatrix}, \quad EWA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1-\eta_w & 0 & 0 \\ 0 & 0 & 1+g_a & 0 \\ 0 & \eta_w & 0 & 1 \end{bmatrix}$$

Finally, we can compute the commutator:

$$[AW, EWA] = AW \cdot EWA - EWA \cdot AW = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -r_a g_a & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

In symbolic form, we can see more clearly how the different parameters interact with one another. For designers, once the direct source of coupling has been identified, they can precisely control the magnitude of the coupling relationship between the two operations by adjusting the specific values of r_a and g_a.

In this section, the commutator we use is actually called an algebraic commutator, which is well-suited for measuring the coupling between two discrete operations. Here, the algebraic commutator's measurement of coupling reflects the order sensitivity of the two operations; it tells us how the coupling between the two operations causes the casting order to produce different effects.

3.4.3 Logical Inverse vs Mathematical Inverse

In addition to reflecting order sensitivity, the algebraic commutator's measure of coupling helps us clearly distinguish between concepts that are easily confused. If we consider a character who is a Feng Shui Master—someone who can not only release but also reclaim—how should we design this reverse skill? For example, the reverse skill for "Aero-Driven-Water" might take the following two forms:

1. Absorb Water Energy and Aero Energy from the environment at the same ratios as the original transfer

2. Completely reverse the effects of the "Aero-Driven-Water" skill, making it appear as though nothing has changed

Option 1 is the intuitive, logical inverse. The flow of resources for the two operations is reversed, but the proportions remain unchanged. Option 2, on the other hand, is a mathematical inverse, meaning it completely cancels out all effects of the previous action. In the actual design process, both Option 1 and Option 2 are reasonable choices. However, at the level of the algebraic commutator, the two actually have significant differences.

Under the first design, the new skill can be expressed as a matrix:

$$AW_{LI} = \begin{bmatrix} 1 & 0 & 0.2 & 0 \\ 0 & 1 & 0 & 0.5 \\ 0 & 0 & 0.8 & 0 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$$

This corresponds to the player absorbing 20% of Aero Energy and 50% of Water Energy from the environment.

The second design is somewhat more complex, and its matrix can be expressed as:

$$AW^{-1} = \begin{bmatrix} 1.25 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ -0.25 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}$$

This result is obtained by taking the inverse of the matrix, and it satisfies the following relationship:

$$AW \cdot AW^{-1} = AW^{-1} \cdot AW = I$$

Here, I represents the identity matrix; if we apply the identity matrix to the game's state vector, the state vector remains unchanged. The prerequisite here is that AW itself is invertible—if its determinant is zero, then no mathematical inverse exists. All subsequent discussions of mathematical inverses in this chapter assume that the matrix is invertible.

This relationship implies that applying the combination of these two operations does not change the game state; in other words, one operation completely offsets the result of the other. In fact, if this skill is applied after AW, we will ultimately return to the initial state. This is a mathematical inverse, meaning it completely offsets all effects of the previous skill. We can analyze the mathematical inverse operation using the algebraic commutator and quickly arrive at:

$$[AW, AW^{-1}] = AW \cdot AW^{-1} - AW^{-1} \cdot AW = 0$$

Their algebraic commutator is 0, meaning there is no coupling between the two operations. This also aligns with the definition of "operations canceling each other out"; regardless of which is cast first, the final result is the same.

However, for the first design, the situation is different. If we use AW first and then the logical inverse skill, the overall effect is as follows:

$$AW_{LI} \cdot AW \cdot s = \begin{bmatrix} 0.84 & 0 & 0.2 & 0 \\ 0 & 0.75 & 0 & 0.5 \\ 0.16 & 0 & 0.8 & 0 \\ 0 & 0.25 & 0 & 0.5 \end{bmatrix} \begin{bmatrix} pa \\ pw \\ ea \\ ew \end{bmatrix} = \begin{bmatrix} 0.84pa + 0.2ea \\ 0.75pw + 0.5ew \\ 0.16pa + 0.8ea \\ 0.25pw + 0.5ew \end{bmatrix}$$

This means that AW and the Logical Inverse skill do not completely cancel each other out in the result. This also aligns with the actual game logic, because after casting AW, the energy in the environment has increased; if we then absorb energy at a fixed rate, we will absorb more energy than we previously consumed.

We can try to calculate their algebraic commutators.

$$[AW, AW_{LI}] = AW \cdot AW_{LI} - AW_{LI} \cdot AW = \begin{bmatrix} -0.04 & 0 & -0.04 & 0 \\ 0 & -0.25 & 0 & -0.25 \\ 0.04 & 0 & 0.04 & 0 \\ 0 & 0.25 & 0 & 0.25 \end{bmatrix}$$

It can be seen that AW and its logical inverse are not commutative; there is a coupling between the two. This fundamentally differs from mathematical inverses, and the coupling introduces a bias in the results due to the order of operations:

$$[AW, AW_{LI}]s = \begin{bmatrix} -0.04(p_a + e_a) \\ -0.25(p_w + e_w) \\ 0.04(p_a + e_a) \\ 0.25(p_w + e_w) \end{bmatrix}$$

This means that swapping the order of the two operations results in a difference in the final outcome between the two. Specifically, if we perform AW_{LI} first and then AW, compared to the reverse

order, the player will suffer an energy loss, while the environment will gain energy. Furthermore, if we calculate the total energy in the result of the commutator, we find that

$$\begin{aligned} E_{total} &= -0.04(p_a + e_a) - 0.25(p_w + e_w) + 0.04(p_a + e_a) + 0.25(p_w + e_w) \\ &= 0 \end{aligned}$$

This means that although the logical inverse operation is not commutative, the total value is conserved. Combining this with the characteristics of the order difference, we can gain further design insights:

"Since AW and the logical inverse are coupled, and since neither changes the total elemental energy regardless of order, by cyclically executing AW and AW_LI, the player can effectively extract energy from the environment at no cost."

Therefore, distinguishing between logical inverses and mathematical inverses is not only important for analyzing mathematical properties but is equally important for the actual design process. If our design goal is symmetry in skill effects, the two skills are in a logical inverse relationship. We cannot expect their effects to completely cancel each other out, and we need to consider the impact of their coupling during the skill's cyclic execution on the game. This can be a feature, or it can be a critical point that causes the design to break down. Consequently, algebraic commutators not only help us uncover this critical insight but also directly aid in the design process. If we need to design a skill akin to "Undo," we must use the mathematical inverse rather than the logical inverse. Furthermore, by adjusting the magnitudes of the terms in the commutator or adding additional constraints, we can keep the extraction effect within a reasonable range, ensuring it functions as a feature rather than a critical bug in the game.

3.4.4 Decomposition of Action Combinations

In our previous analysis, we measured coupling by comparing the differences resulting from different sequences of skill execution. However, coupling is an intrinsic property between operations—even when the AW.EWA combination is executed just once, the effects of coupling are already embedded in the results. So, can we directly decompose a single AW.EWA combination into two parts—"ideal effect" and "coupling deviation"—to investigate the impact of coupling within a single action combination? This has more direct practical significance for more practical game analysis.

Recall the results when the two skills act in different orders:

$$AW \cdot EWA \cdot s = \begin{bmatrix} 0.8pa \\ 0.35pw \\ 1.4ea + 0.2pa \\ ew + 0.65pw \end{bmatrix}, \quad EWA \cdot AW \cdot s = \begin{bmatrix} 0.8pa \\ 0.35pw \\ 1.4ea + 0.28pa \\ ew + 0.65pw \end{bmatrix}$$

Here, the player's energy and the environment's Water Energy are exactly the same under different orders, but the environment's Aero Energy is $1.4ea + 0.2pa$ under AW.EWA and $1.4ea + 0.28pa$ under EWA.AW. So, intuitively, can we conclude that the coupling causes a shift in the environment's Aero Energy in the final result? At the same time, the coupling does not cause any shift in the other energy components, so these energies are identical for both operation orders. If this intuition is verified, can we interpret a single combined operation as an "ideal" result plus a coupling perturbation?

In fact, matrix operations and the algebraic commutator provide a very good answer to our expectations.

For the combination of single operations AB, mathematically we can perform the following identity transformation

$$AB = \frac{AB+BA}{2} + \frac{AB-BA}{2} \tag{3.5}$$

$$= \underbrace{\frac{1}{2}\{A,B\}}_{\text{Average Effect}} + \underbrace{\frac{1}{2}[A,B]}_{\text{Coupling Effect}} \tag{3.6}$$

where $\{A,B\} = AB+BA$, which is mathematically known as the anti-commutator and represents an averaging effect.

Through this identity transformation, the single-operation combination is decomposed into two parts. Similarly, we can decompose BA to obtain the decomposition forms for the two operation sequences:

$$AB = \frac{1}{2}\{A,B\} + \frac{1}{2}[A,B] \tag{3.7}$$

$$BA = \frac{1}{2}\{A,B\} - \frac{1}{2}[A,B] \tag{3.8}$$

In a single composition operation, the effect of this operation can be decomposed into two parts: the average effect represented by the anti-commutator and the coupling effect represented by the commutator. Furthermore, for the two orders, the average effect remains consistent, while the coupling effect exhibits a positive and negative deviation. This means that in a single composition operation, under different orders, the coupling effect causes us to deviate from the average effect in different directions. This is exactly what we expect: to investigate the impact of the coupling effect in a single operation.

We can observe the specific expansion of this expression in the case of AW.EWA

$$AW \cdot EWA = \frac{1}{2}(AW \cdot EWA + EWA \cdot AW) + \frac{1}{2}(AW \cdot EWA - EWA \cdot AW) \tag{3.9}$$

$$= \underbrace{\begin{bmatrix} 0.8 & 0 & 0 & 0 \\ 0 & 0.35 & 0 & 0 \\ 0.24 & 0 & 1.4 & 0 \\ 0 & 0.65 & 0 & 1 \end{bmatrix}}_{\text{Average Effect}} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -0.04 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{\text{Coupling Effect}} \tag{3.10}$$

Observing the coupling effect section, only the pa-ea term is non-zero. This once again confirms our earlier intuition: coupling causes a shift in the environmental Aero Energy in the final result. Furthermore, under the identity transformation, we gain a new interpretation of the commutator matrix elements:

An element $[A,B]_{ij}$ in the i-th row and j-th column that is non-zero indicates the existence of a unidirectional coupling channel from parameter j to parameter i between the two operations. This means that this coupling channel influences, within the combined operation, the extent to which parameter j causes parameter i to deviate from the average effect. The magnitude of this deviation is $\frac{1}{2}[A,B]_{ij}$.

Ultimately, through this identity transformation, we not only gain the ability to analyze the coupling effects between operations in single-operation combinations but also gain a deeper understanding of how coupling affects the results of operations even without direct comparisons of different operational sequences. At the same time, based on the anti-commutator and commutator combination, we can gain further insights for potential designs. When we calculate the anti-commutator $\{AW,EWA\}$, we can understand the baseline that the two skills will always possess, regardless of the order in which they are cast. Calculating the commutator, on the other

hand, clarifies how different orders cause the results to deviate from the baseline. For AW.EWA, by comparing the number and relative magnitude of the matrix elements, we can see that the average effect is significantly greater than the coupling effect. Therefore, the relationship between these two skills is: the overall combined effect is relatively consistent, but the coupling effect causes slight deviations depending on the casting method. In design terms, this corresponds to skill combinations that perform consistently; the differences in coupling effects are merely a bonus for decision-making. Players may not need to spend much time considering the impact of coupling effects. Conversely, if we want to design a difficult choice that requires players to expend significant mental effort, from this perspective, our operations might have the following characteristics: the average effect is comparable to or even weaker than the coupling effect. This makes coupling have a significant impact on the outcome, requiring players to carefully analyze the coupling effects.

3.5 Generalization

The preceding analysis focused on the coupling between two operations, but for our Feng Shui Master, two skills are clearly insufficient. Therefore, if we introduce a new skill for the Feng Shui Master:

Feng Shui Cycle (WAC): Absorbs 10% of the Water Energy in the environment and converts it into the character's own Aero Energy, while simultaneously releasing 10% of the character's own Aero Energy and converting it into Water Energy in the environment.

For this new skill, we can separately consider the commutators of the new skill with Aero-Driven-Water and with Water-Induced-Aero, applying all the methods used previously. Furthermore, when we treat AW.EWA as a composite operation, the relationship between the new skill C and the original skills can similarly be analyzed via $[WAC, AW.EWA]$. This leads to an analysis of coupling between operations and combinations; in the case study of the next chapter, we will see a concrete application of this extension.

3.6 Declaration: Scope of Linearity

Throughout the analysis in Chapter 3, we have consistently represented game operations as matrices and used the algebraic commutator $[A,B] = AB - BA$ to measure coupling. This framework relies on a core premise: the effects of operations are linear and independent of the current state.

Linearity means that the effect of an action can be represented by a fixed matrix. Diagonal elements represent scaling factors for attributes, off-diagonal elements represent conversion ratios between attributes. This representation is natural for most game actions—increasing attack power, consuming mana, converting defense into attack, and so on—all of which can be precisely expressed as matrices.

However, not all game operations are linear. Condition-triggered effects (such as "If health is below 50%, attack power doubles"), threshold checks, and nonlinear functions (such as "Apply a bonus after taking the logarithm of attack power") cannot be represented by a single fixed matrix. These operations fall under the category of state-dependent actions and require more complex mathematical representations, such as parameterized families of matrices or piecewise linear functions. The analysis of these state-dependent actions constitutes a future research direction.

All operations analyzed in this chapter and subsequent case studies are assumed to be linear and state-independent. This means that the results of the commutator capture the structural coupling of the operations themselves, rather than their behavior under specific states. This assumption defines both the scope of this framework and its core advantage: precisely because operations are represented as fixed matrices, the commutator can provide a measure of coupling that is independent of any game state and depends solely on the structure of the operations.

4. Case Study - Chaos Fighter

To further demonstrate how the commutator can be applied to specific game analysis, we will examine a local system within a concrete game prototype. Here, the description of traditional testing methods is based on widely accepted design process knowledge in the field of game design, such as testing and iteration. The comparison between commutator analysis and traditional testing focuses on the fundamental differences between the two in terms of the types of information output and acquisition efficiency, rather than an empirical evaluation of designers' actual work efficiency.

4.1 Game Introduction

Chaos Fighter is an auto-battler game designed for casual family entertainment or social gatherings. All players' characters battle in an arena until a single victor remains. Due to the chaotic nature of combat, matchups within the arena are entirely random. Therefore, to secure victory, players must build their characters' abilities through deck construction before entering the arena. Over multiple rounds of combat, the player who ranks first in the most rounds is awarded Grand Champion, and the player who wins the final round is awarded Ultimate Champion, respectively.

4.1.1 Design Motivation

The auto-battler-style design of character-building and automatic combat found in Chaos Fighter is prevalent in current commercial games. Whether it's Teamfight Tactics in League of Legends or Hearthstone's Battlegrounds, these are successful and influential auto-battler gameplay modes. Auto-battlers are naturally well-suited for validating our analytical methods. For players, the deck-building phase is the only controllable factor. Therefore, the coupling between any two actions a player performs during deck-building will influence the final outcome of the game. Coupling analysis is crucial during the deck-building phase of auto-battler. At the same time, compared to other game genres, such as multiplayer competitive shooters or real-time strategy games, Auto-battler isolates direct, turn-based interactions between players that influence the game's outcome. This means that by using auto-battler as a case study, we can focus on analyzing the coupling relationships within a single player's actions, making the results clearer and easier to interpret.

4.1.2 Game Rules

1. Drafting Phase: Players may select any 8 cards from the Basic deck and 2 cards from the Unique deck, then shuffle the cards.

2. Preparation Phase: Players draw 4 cards and may discard and redraw up to 2 cards. If the draw pile is empty, shuffle the cards from the discard pile and return them to the draw pile.

3. Action Phase: Players play any cards in any order and choose one of four combat strategies.

4. Combat Phase: All player characters enter the arena and are randomly paired for combat; each battle consumes 10 Stamina points.

5. Resolution Phase: When only one player remains, combat ends and resolution begins. The round counter increases by 1, and all players adjust their character status based on attribute growth and health recovery. If this is the final round, the game ends.

6. Deck-Building Phase: Draw a number of cards equal to the number of players plus one from the special card pile, displaying them in a 1:1 ratio of face-up to face-down cards. Starting with the winner of the previous round, players select cards; players may also choose not to take any cards. Additionally, the winner of the previous round may choose to either 1. take two cards or 2. view all face-down cards. The newly added cards are placed at the bottom of the player's draw pile. After all players have selected cards, any remaining cards are placed at the bottom of the special card pile. The game then proceeds to the next Preparation Phase.

In the rules described above, the Action Phase is the only stage where players can actually influence their character's state. Therefore, our subsequent analysis will focus on the interactions between cards during this phase.

Compared with Current Methods

Existing automated game analysis methods—such as Monte Carlo tree search, Markov decision processes, and statistical learning—largely rely on the collection of large amounts of data or large-scale simulations. These methods essentially simulate and analyze the game process; they can answer questions like "What is this deck's win rate?" or "What is the optimal play sequence?", but they cannot directly answer "Is there a coupling between these two cards? Between which parameters does the coupling occur? What determines its strength?" The commutator analysis we are about to explore offers a complementary perspective: it does not rely on simulation data but instead starts from the structure of card effects themselves, revealing coupling relationships, strength, and the origins of parameters.

4.2 Game State and Card Pool

Next, we will introduce the game state and card pool, and use the methods from Chapter 3 to express them algebraically.

4.2.1 Game State Representation

A player's state consists of eight parameters: the four base attributes—Attack, Defense, Stamina, and Health Points (HP)—and their corresponding growth values. We can represent this as:

$$s = [ATK, DEF, STA, HP, GATK, GDEF, GSTA, GHP, 1]^T$$

This representation essentially flattens the previously vertical parameters; mathematically, this is called a transpose. We use this notation when dealing with a large number of parameters. It is worth noting that the final number 1 is known mathematically as a homogeneous coordinate. If we wish to perform numerical addition in matrix operations (which is extremely common in games), homogeneous coordinates are necessary. We will verify this in subsequent analyses.

With this single set of parameters, we can fully describe the state of a player character in Chaos Fighter.

4.2.2 Card Pool

The game's card pool is designed as a combination of basic cards and unique cards. Basic cards provide one-dimensional numerical bonuses to attribute values and attribute growth; there are eight of them in total. Unique cards, on the other hand, apply more complex modifications to attributes, significantly impacting a character's state.

We have designed four unique cards:

Combination TrainingM_1: Stamina +20%, Stamina Growth +5, Attack and Defense -5%, HP -10

Combination Training is designed to provide a stamina boost at a cost.

Swords Made from ArmorM_2: Converts 30% of existing Defense into Attack; Stamina -20

Swords Made from Armor represent a traditional direct conversion of Defense to Attack, suitable for players with high Defense and low Attack strategies.

Stealing the FutureM_3: Immediately resolves attribute growth twice; all subsequent attribute growth -3

Stealing the Future embodies the concept of "stealing the future." It allows players to gain immediate benefits but reduces long-term gains.

Original CombustionM_4: HP (10% + 50), Attack (20% + 10), but Defense -20% and HP Growth -10

Original Combustion is an offensive burst card that boosts attack power while weakening defense and recovery capabilities.

Compared to basic cards, these four unique cards have a more significant impact on character builds. Players must carefully weigh the benefits against the costs when using them.

4.2.3 Card Matrix Representation

Taking Combination Training as an example, we derive the matrix representation of a card based on its effects, following the rules of matrix operations discussed earlier.

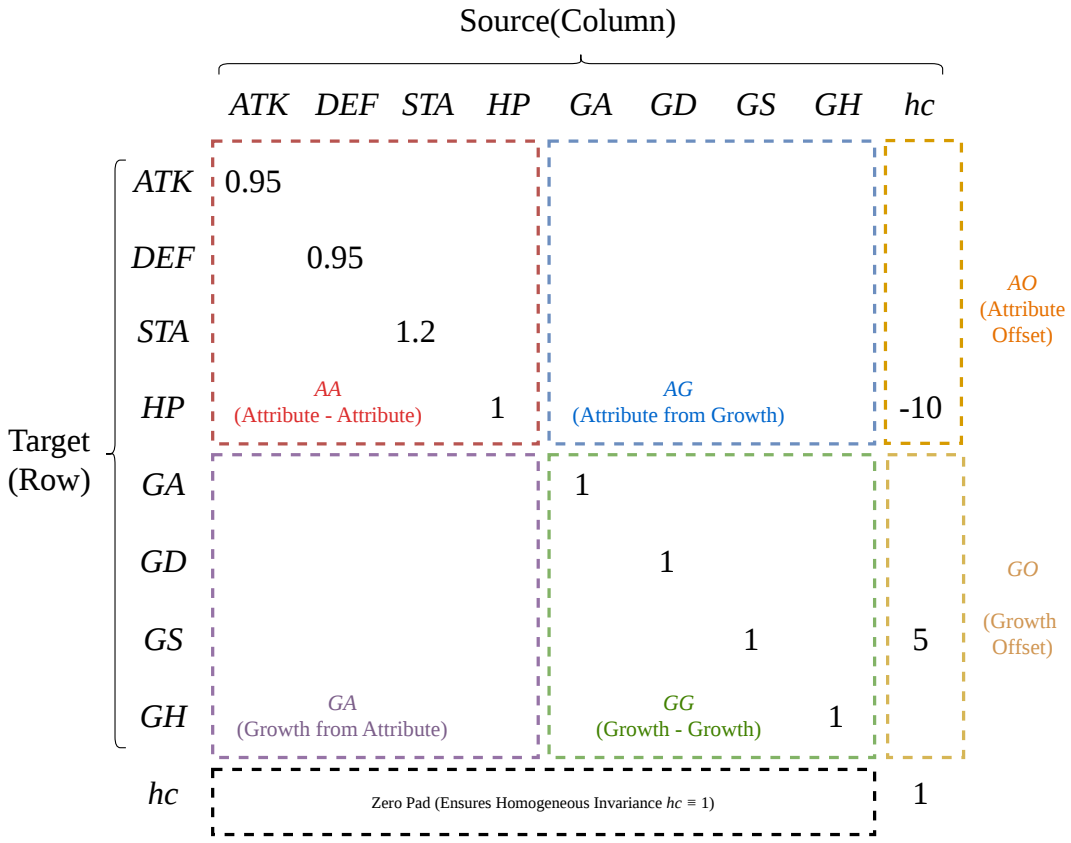


Figure 4.1. Matrix Representation of Combination Training M_1

In Figure 4.1, we divide the card's effects into six major blocks: Attribute-Attribute, Attribute-Growth, Growth-Attribute, Growth-Growth, Attribute Offset, and Growth Offset. By analyzing these distinct blocks, we can quickly grasp the card's overall characteristics.

AA: Elements on the diagonal that are not 1 indicate a proportional change in attribute values. The absence of off-diagonal elements suggests there is no conversion relationship between attribute values.

AG, GA: All elements are 0, indicating that this card has no effects involving both attribute and growth values.

GG: All diagonal elements are 1, indicating that growth values remain constant in the proportional relationship.

AO, GO: Each has one non-zero element; the elements in the homogeneous columns indicate that this card affects attribute values and attribute growth by a fixed amount.

Thus, we can see that even before performing commutator analysis, the matrix representation already provides a wealth of design information.

In fact, partitioning is a common mathematical practice; we can represent the operation matrix for all cards as:

$$\mathbf{M} = \begin{bmatrix} \mathbf{M}_{AA} & \mathbf{M}_{AG} & \mathbf{V}_{AO} \\ \mathbf{M}_{GA} & \mathbf{M}_{GG} & \mathbf{V}_{GO} \\ \mathbf{0}_{1\times 4} & \mathbf{0}_{1\times 4} & 1 \end{bmatrix} \tag{4.1}$$

We can then write out the expressions for each submatrix, for example:

$$\mathbf{M}_{AA} = \begin{bmatrix} 0.95 & 0 & 0 & 0 \\ 0 & 0.95 & 0 & 0 \\ 0 & 0 & 1.2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{V}_{AO} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -10 \end{bmatrix} \tag{4.2}$$

By combining the target-source relations for each block, we can conduct a detailed discussion on a block-by-block basis without having to use a large 9×9 matrix. To maintain clarity and readability in the main text, this chapter presents only the results for non-zero entries of the commutator in each block. The complete 9×9 matrices for all cards and the full calculation results for the commutator are included in Appendix A. Additionally, following the conventions in the figure, we will represent the positions of non-zero matrix elements in the form of attribute pairs. For example, for the matrix element $a_{34} = 0$ in the third row and fourth column, we will write it directly as $(STA, HP) = 0$, indicating that Stamina is not affected by Health Points during this operation. On the one hand, this reduces the reader's cognitive load, and on the other, it clarifies the design intent.

With this, we have established the state space and the matrix representation of unique cards. Next, we will first focus on the first three cards (Combination Training M_1, Swords Made from Armor M_2, and Stealing the Future M_3) to reveal the coupling structure among them through commutator analysis.

4.3 Commutator Analysis for Existing Cards

In this section, we will first calculate and analyze the commutators between individual cards to examine their coupling relationships. Subsequently, we will analyze the three-card commutators as a whole, exploring the statistical insights they can offer us.

4.3.1 Combination Training - Swords Made from Armor

The commutator $C_{1,2} = [M_1, M_2] \neq 0$, indicating coupling.

The only non-zero term is $(STA, hc) = -4$. The homogeneous coordinate hc indicates that Stamina is not influenced by other attribute values but rather represents a fixed-value deviation. Therefore, the coupling between these two operations yields a fixed effect under all circumstances, regardless of the character's current state. In Chapter 3, the Feng Shui Master's analysis of the

element $(pa, ea) = -0.08$ in the commutator $[AW, EWA]$ indicates that a player's wind energy is influenced by environmental wind energy within the coupling. In contrast, $(STA, hc) = -4$ is more stable and represents a "well-behaved" coupling relationship in design.

Compare this to traditional methods: designers might intuitively assume that the stamina consumption of "Combination Training" is affected by the stamina reduction from "Swords Made from Armor." However, intuition struggles to distinguish whether "this effect is fixed or proportional," and this distinction is crucial in design. A fixed offset means the coupling effect remains constant regardless of the character's current state; a proportional offset, however, will amplify or diminish as the character's attributes change. The commutator, through non-zero terms located in homogeneous columns rather than AA blocks, provides a precise answer: this is a fixed-value, stable coupling.

4.3.2 Swords Made from Armor - Stealing the Future

For the commutator $C_{2,3} = [M_2, M_3] \neq 0$, coupling exists.

Non-zero terms: $(ATK, GDEF) = 0.6$, $(DEF, GDEF) = -0.6$. Both non-zero terms are located in the AG block, representing a coupling relationship between attributes and growth effects. This differs significantly from the constant bias observed in $C_{1,2}$.

For the non-zero terms, the source of the parameter coupling is defense growth $GDEF$, but the affected attributes differ: attack and defense, respectively. More interestingly, while their numerical values are identical, their signs are opposite. Combined with the operational decomposition equation 3.7, this coupling relationship can be further interpreted. Under the sequence $M_2.M_3$—where Stealing the Future takes place first, followed by Swords Made from Armor—the character's attribute values will have slightly more attack power and slightly less defense power than the average effect. Conversely, for the opposite sequence, the signs of the non-zero terms will also reverse, and the character will gain more defense power and less attack power. Therefore, in design terms, we can interpret this as a trade-off: given the same defense growth value, do you prefer to gain more attack power or more defense?

Compare this to the traditional approach: through testing, designers can observe that "Stealing the Future and then creating Swords Made from Armor result in higher attack power; the reverse yields higher defense." However, traditional testing struggles to directly reveal a key fact: the numerical values of the attack power gain and the defense loss are exactly equal, differing only in sign. This implies that this is a precisely symmetrical trade-off. The fact that the absolute values of $(ATK, GDEF) = 0.6$ and $(DEF, GDEF) = -0.6$ in the commutator are equal directly proves this. Furthermore, the parameterized commutator can inform designers: if the defense growth value or conversion ratio is adjusted, how will the equilibrium point of this trade-off shift?

4.3.3 Combination Training - Stealing the Future

For the commutator $C_{1,3} = [M_1, M_3] \neq 0$, coupling exists. Non-zero terms: $(ATK, GATK) = -0.1, (DEF, GDEF) = -0.1, (STA, GSTA) = 0.4, (STA, hc) = -10$. The non-zero terms are distributed across the AG and AO blocks, exhibiting both attribute-growth coupling and attribute-constant deviations.

For the non-zero terms within the AG block, we observe that the couplings all involve attributes being influenced by their corresponding growth values; that is, there is no cross-attribute coupling. Compared to attack and defense, the coupling has a greater impact on stamina. Stamina is influenced not only by stamina growth but also by a constant bias. Furthermore, we observe that the signs of $(STA, GSTA)$ and (STA, hc) are opposite, indicating that the coupling affects Stamina in both directions. Under different activation sequences, Stamina is subject to both a positive and a negative effect. This means that when considering the combined effects of these two cards, players must not only account for the relationships between Attack, Defense, and Stamina, but also weigh the outcome of Stamina itself under these two opposing effects. Therefore, compared

to the combination with "Swords Made from Armor - Stealing the Future," players face a greater number of complex trade-offs with these two cards. This also implies that adjustments to these two cards require greater caution, as they involve two balancing points. Any numerical change could lead to significant alterations in the interactions between the cards.

Compared to traditional methods: These two cards involve the combined effects of current attributes and growth attributes, and simultaneously include both proportional conversion and fixed-value effects. Under traditional methods, designers must conduct multiple rounds of testing, observe results under different initial states, and then manually separate the contributions of the "proportional component" and the "fixed-value component." Alternatively, they must clearly distinguish between the two based on experience during numerical analysis. However, the commutator, through the partitioned distribution of non-zero terms in the AG blocks and homogeneous columns, directly separates these two types of coupling effects without the need for testing or manual separation.

More importantly, the fact that the signs of $(STA, GSTA) = 0.4$ and $(STA, hc) = -10$ are opposite is immediately apparent in the commutator. In traditional testing, this "opposite sign" may be obscured by the total change in the Stamina dimension. This is because testing can only observe the net result and cannot automatically distinguish between "positive growth coupling" and "negative fixed-value coupling." The clear visualization of coupling relationships between each parameter in the commutator allows designers to better understand the complex interrelationships within the coupling.

4.4 Commutator Analysis for Adding a New Card

$C_{1,2}$, $C_{2,3}$, and $C_{1,3}$ establish three distinct types of coupling structures among three cards. These analyses are based on a closed three-card system. However, in actual gameplay, the card pool is dynamic—the addition of new cards may alter the existing dynamics. Next, we will introduce a fourth card, "Original Combustion," and analyze how it impacts the existing three-card system.

4.4.1 Single - Single Commutator

We can illustrate the direct relationship between the new card and the existing cards using a table.

Table 4.1. *Non-zero commutator terms between the new card and the existing cards*

Commutator	Non-zero terms (target, source)	Block
$C_{1,4}$	(ATK, hc) = -0.5, (HP, hc) = 1	Homogeneous Column
$C_{2,4}$	(ATK, DEF) = -0.12	AA Block
$C_{3,4}$	(ATK, GATK) = -0.4, (DEF, GDEF) = 0.4	AG Block

Table 4.1 summarizes all commutator results between Original Combustion and the three legacy cards. Each commutator can be obtained with a single matrix operation. We do not need to design test cases for each card pair, execute operations under different initial states, and compare results as in traditional testing. Furthermore, if designers wish to adjust the parameters of "Original Combustion" (for example, changing the attack bonus from 20 % to 30%), they need only modify the corresponding element in the matrix and recalculate the commutator. Parametric analysis compresses the "adjustment-verification" cycle from multiple rounds of actual testing down to a single matrix operation, offering a significant efficiency advantage in analyzing coupling relationships.

4.4.2 Single - Combined Commutator

As the game progresses, players acquire more unique cards. Therefore, during a single action phase, a player may play two or more unique cards. The coupling relationship between two

cards becomes somewhat obscured when multiple cards are played. We previewed the analysis of combination operations in Chapter 3. Next, we will delve into a deeper analysis: the coupling relationship between Original Combustion and combinations with older cards. This is a natural extension of single-to-single coupling analysis and represents the most unique value of the commutator relative to traditional testing.

Combination Training—The "Swords Made from Armor" Combination

In traditional testing, we typically consider the overall impact of a new card on existing cards. However, the commutator explicitly indicates that when two cards are coupled, different orders yield different results. Therefore, the commutator $C_{1,2} \neq 0$ requires us to analyze card combinations by case. This high sensitivity to order—or coupling—is not only a fundamental requirement of commutator analysis but also provides designers with a clearer perspective.

We calculated the commutators for "Original Combustion" and two different order combinations; the results are shown in the table below.

Table 4.2. *Commutator: M_4 with M_1, M_2*

Commutator	Block	Non-zero Term Location	Value
$C_{4,12} = [M_4, M_1M_2]$	AA	(ATK, DEF)	0.114
	AO	(ATK, hc)	0.5
	AO	(HP, hc)	-1
$C_{4,21} = [M_4, M_2M_1]$	AA	(ATK, DEF)	0.114
	AO	(ATK, hc)	0.5
	AO	(HP, hc)	-1

From the table, we can see that the coupling between the combination effects of the new and old cards is concentrated in AA (between attribute values) and AO (fixed numerical deviations in attribute values). More importantly, $C_{4,12} = C_{4,21}$. This directly highlights a crucial fact: the coupling between Original Combustion and the "Combination Training + Swords Made from Armor" combo is insensitive to the internal order of the combo (i.e., the sequence in which the cards are played). Regardless of the order in which Combination Training and Swords Made from Armor are played, the overall coupling between Original Combustion and the two cards remains the same. In traditional testing, designers typically focus only on the "overall impact" of a new card on existing card combinations, rarely delving into whether "the internal order of the combination alters the coupling effect of the new card." To reach the same conclusion through testing, one would need to test the effect of Original Combustion under both internal orders while strictly holding all other variables constant. This additional cost means that such analysis is rarely performed in standard design processes.

Combination Training—Stealing the Future Combination

Similarly, we calculated the commutators for two skill combinations, with the results shown in the table below.

Table 4.3. *Commutator: M_4 with M_1, M_3*

Commutator	Block	Non-zero Term Location	Value
$C_{4,13} = [M_4, M_1, M_3]$	AG	$(ATK, GATK)$	0.38
	AG	$(DEF, GDEF)$	-0.38
	AO	(ATK, hc)	0.5
	AO	(HP, hc)	-1
$C_{4,31} = [M_4, M_3M_1]$	AG	$(ATK, GATK)$	0.4
	AG	$(DEF, GDEF)$	-0.4
	AO	(ATK, hc)	0.5
	AO	(HP, hc)	-1

Compared to the combination of the previous two skills, the combination of Combination Training and Stealing the Future exhibits more subtle changes. The two commutators have identical non-zero terms in the AO block, while a very slight deviation appears in the AG block. For the two different card combination sequences, the terms $(ATK, GATK)$ and $(DEF, GDEF)$ differ by 0.02. This indicates that the coupling between Original Combustion and the combination of Combination Training + Stealing the Future creates a slight sensitivity to the order in which they are cast. This creates a subtle yet fundamental difference from the case where $C_{4,12} = C_{4,21}$. The overall result of "Original Combustion" combined with "Combination Training" and "Swords Made from Armor" depends not only on the order of "Original Combustion" relative to the other two skills, but also on the internal order of the two skills themselves. Regarding this 0.02 deviation, we can precisely pinpoint its source through symbolic computation.

Let m_{1ad} be the percentage reduction in Attack Power from Combination Training (5%), m_{3gr} be the multiplier for Stealing the Future (2), and m_{4ar} be the percentage increase in Attack Power from Original Combustion (20%). Algebraic calculations show:

$$C_{4,13}(ATK, GATK) = (1 - m_{1ad}) \cdot m_{3gr} \cdot m_{4ar} = 0.95 \times 2 \times 0.2 = 0.38$$

$$C_{4,31}(ATK, GATK) = m_{3gr} \cdot m_{4ar} = 2 \times 0.2 = 0.4$$

The difference between the two is $m_{3gr} \cdot m_{4ar} \cdot m_{1ad} = 0.02$. This indicates that the 0.02 difference is the precise result of a three-way parameter interaction: the attack power reduction effect from Combination Training is carried over during the Stealing the Future resolution process, and interacts with the attack power gain from Original Combustion. When Combination Training resolves before Stealing the Future, the reduction effect takes effect before resolution, and the gain from Original Combustion is applied to the already reduced attack power; when the order is reversed, the gain acts on the full attack power. Although this difference is minute, its parametric origin is fully transparent within the matrix structure. Therefore, the symbolic capability of the commutator allows designers to employ precise parameter-level control methods. Traditional testing not only struggles to detect this difference but also finds it even more difficult to precisely attribute it to the product of the three parameters and perform fine-tuning.

The "Swords Made from Armor - Stealing the Future" Combination
This is the final possible commutator, and our calculation results are as follows:

Table 4.4. *Commutator: M_4 with M_2 and M_3*

Commutator	Block	Non-zero Term Location	Value
$C_{4,23} = [M_4, M_2M_3]$	AA	(ATK, DEF)	0.12
	AG	$(ATK, GATK)$	0.4
	AG	$(ATK, GDEF)$	0.12
	AG	$(DEF, GDEF)$	-0.28
$C_{4,32} = [M_4, M_3M_2]$	AA	(ATK, DEF)	0.12
	AG	$(ATK, GATK)$	0.4
	AG	$(DEF, GDEF)$	-0.4

This is the most complex case. The commutators differ not only numerically but also in their non-zero terms. Compared to $C_{4,32}$, $C_{4,23}$ has an additional term: $(DEF, GDEF)$. This indicates that the combination order of "Swords Made from Armor" and "Stealing the Future" not only affects the magnitude of the coupling with Original Combustion but also influences the form of the coupling.

We can also explore this further by reducing it to a symbolic form.

Let $m_{4ar} = 0.2$ be the attack gain ratio of Original Combustion, $m_{2da} = 0.3$ be the defense conversion ratio of Swords Made from Armor, and $m_{3gr} = 2$ be the growth growth resolution multiplier of Stealing the Future. When "Stealing the Future" is played before "Swords Made from Armor," M_2 and M_3 generate a non-zero term $m_{2da} \cdot m_{3gr}$ at the $(ATK, GDEF)$ position. This

represents the GDEF $\rightarrow$ DEF $\rightarrow$ ATK path formed when GDEF is settled as DEF via “Stealing the Future” and then converted to ATK by “Swords Made from Armor”. The gain coefficient $1+m_{4ar}$ of Original Combustion interacts with this path, forming a coupling:

$$C_{4,23}(ATK, GDEF) = m_{4ar} \cdot m_{2da} \cdot m_{3gr} = 0.2 \times 0.3 \times 2 = 0.12$$

When the order is reversed (first using Card Swords Made from Armor, then using "Stealing the Future"), M_3 and M_2 are zero at the $(ATK, GDEF)$ position. In this case, the resolution result of "Stealing the Future" does not directly generate a conversion path to ATK in the combination matrix, so $C_{4,32}(ATK, GDEF) = 0$. This coupling term completely disappears in the reverse order.

For designers, this is more complex than the previous two cases and is also easier to overlook. Without explicitly classifying the order of card combinations, this coupling effect—which depends on the order in which cards are played—may be subtle and difficult to detect during testing. Only when "Stealing the Future" is played before "Swords Made from Armor" will the test reflect the $(ATK, GDEF)$ coupling relationship.

Consequently, on the one hand, such complex coupling relationships can become blind spots in design, leading to unexpected outcomes under specific play sequences. On the other hand, they can also serve as a source of advanced gameplay strategies. Players may learn to master this coupling through gameplay, ultimately enabling them to accumulate small advantages and secure victory. Designers can also consciously design and leave room for such complex coupling, anticipating that players will explore and delve deeper into it.

4.4.3 Nested Commutator

The commutator of a combination card reveals the sensitivity of a single card to the internal order of the combination card. In fact, we can further use the nested commutator to quantify this sensitivity. The relationship between the nested commutator and the combination commutator is as follows:

$$[C,[A,B]] = [C,AB] - [C,BA] \tag{4.3}$$

Table 4.5. *Correspondence between the nested commutator and the first-order combinatorial commutator*

Nested Commutator	Non-zero Terms and Values
$[M_4,[M_1,M_2]]$	None
$[M_4,[M_1,M_3]]$	$(\mathrm{ATK},\mathrm{GATK}) = -0.02$, $(\mathrm{DEF},\mathrm{GDEF}) = 0.02$
$[M_4,[M_2,M_3]]$	$(\mathrm{ATK},\mathrm{GDEF}) = 0.12$, $(\mathrm{DEF},\mathrm{GDEF}) = 0.12$

The numerical value of the nested commutator directly reflects the interaction strength between M_4 and existing coupling channels. If the nested commutator is zero, it indicates that M_4 does not interact with the internal coupling of that combination, and the designer need not pay further attention to it. If the nested commutator is non-zero, the designer can trace back to the corresponding first-order combination commutator to confirm whether the specific manifestation is a "fine-tuning of strength" or the "presence or absence of a channel."

There is no equivalent method for this level of analysis in traditional testing. As a design analysis tool, nested commutators can be automatically calculated by a program, and the results can guide designers in conducting targeted, precise diagnostics of first-order combination commutators. This is precisely where the engineering potential of the commutator analysis framework lies.

4.4.4 Summary

Through the above analysis, we have demonstrated how, following the introduction of Original Combustion, commutator analysis can systematically evaluate the impact of new cards on the existing card ecosystem.

At the first level (pairwise commutator analysis), the commutator analysis rapidly identified the coupling types between Original Combustion and three existing cards: fixed-value coupling with Combination Training, intra-attribute coupling with Swords Made from Armor, and attribute-growth coupling with Stealing the Future. These three coupling types are automatically distinguished during computation, eliminating the need for individual testing.

At the second level (combo commutator analysis), commutator analysis reveals information that traditional testing can hardly access: the sensitivity of the coupling between Original Combustion and old card combos to the internal order of the combo. The three combinations exhibit a clear hierarchy of sensitivity:

The coupling between Original Combustion and other cards' combinations is:

1. Completely insensitive to the internal order of Combination Training + Swords Made from Armor;

2. Slightly sensitive to the internal order of Combination Training + Stealing the Future (a 0.02 difference in the AG block);

3. Significantly sensitive to the internal order of Swords Made from Armor + Stealing the Future (a difference in the presence or absence of the coupling channel).

Nested commutators further trace this sensitivity precisely to the product of three parameters.

The key distinction from traditional methods lies in the fact that all this coupling information is obtained through matrix calculations before a new card is introduced into the game. Designers can predict changes in the card pool's coupling structure upon the addition of Original Combustion during the design phase. Which existing combinations will be affected, in which parameter channel the impact occurs, and the magnitude of the impact— none of this requires waiting for player test feedback or conducting large-scale match simulations.

4.5 Statistical Commutator Analysis

In the preceding analysis, we examined different types of commutators between cards. Each commutator precisely reveals coupling information for a specific card pair or combination. For example, $[M_2, M_3]$ tells us that "Swords Made from Armor" and "Stealing the Future" are coupled in the attribute-growth segment; $C_{4,23}$ tells us that the coupling between "Original Combustion" and the combination of "Swords Made from Armor" and "Stealing the Future" varies depending on the internal order.

However, each commutator provides only a local perspective. It concerns only specific pairs or triplets of cards. Designers can, of course, read through the results of all commutators one by one, but as the card pool expands, the number of commutators grows with the number of combinations. More importantly, reading them one by one cannot directly reveal the structural characteristics of the entire card pool: Which attribute dimensions frequently exhibit coupling? Which dimensions never exhibit coupling? Which cards are coupling hubs, and which are coupling islands?

Although different commutators involve different combinations of operations, all commutators share the same matrix form and rules for interpreting matrix elements. Rows represent targets, columns represent sources, and the positions of non-zero entries indicate coupling channels. Building on this foundation, we can perform block-wise statistics on the non-zero matrix elements of commutators for all card combinations, summarizing them into a global coupling distribution. This statistical perspective is not only necessary for large card pools, but also reveals the overall structure of our current small card pool—a structure that cannot be directly grasped by reading commutator results one by one.

4.5.1 Coupling Strength Heatmap

By taking the absolute values of all elements in the commutators between cards and summing them, we obtain a special matrix C_{AbsSum}:

$$C_{AbsSum} = \begin{bmatrix} 0 & 0.12 & 0 & 0 & 0.5 & 0.6 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 1.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.4 & 0 & 14 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{4.4}$$

The reason for taking the absolute value is that matrix elements at the same position for different commutators may have opposite signs and could cancel each other out when summed. If we want to measure the distribution of overall coupling strength, taking the absolute value ensures that all coupling terms are reflected.

From C_{AbsSum}, we can get a preliminary sense of which parameters are coupled, with the overall strength directly reflected by the numerical values. For example, $(DEF, GDEF) = 1.1$ indicates the strongest coupling among all parameters. This suggests that, in our current design of four cards, there is significant interaction between cards where defense is influenced by defense growth. $(STA, hc) = 14$ indicates that interactions between cards result in significant fixed-value deviations in Stamina.

However, the matrix is relatively less intuitive and not very straightforward for designers. Therefore, we can plot a heatmap based on C_{AbsSum}, as shown in 4.2.

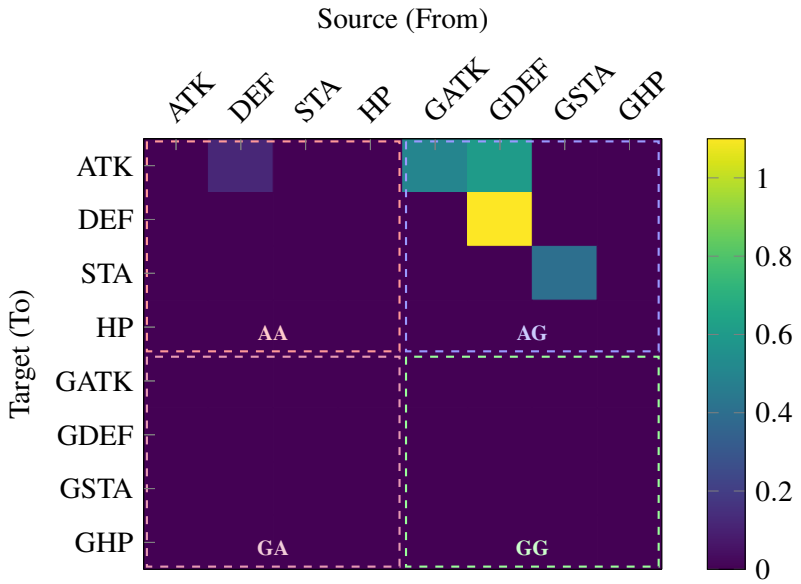


Figure 4.2. Heatmap of the strength of non-zero terms among commutators

Since the values for the homogeneous columns AO and GO differ significantly from those in the other blocks, and since homogeneous columns are better suited for separate analysis, we have excluded the homogeneous-related sections from the heatmap. Compared to the mathematical matrix form, the heatmap is more intuitive and clear. By examining Figure 4.2, we can immediately draw several general conclusions, such as

- Overall distribution characteristics: Interactions among existing cards are concentrated in the AG zone, with only a few in the AA zone, while the GA and GG zones show none at all.

This indicates that the current deck design causes card interactions to be highly concentrated on the influence of growth values on attribute values. Couplings between growth attributes and those based on attribute values affecting growth values are completely absent.

- Distribution characteristics within zones: Within the AG zone, coupling relationships where Attack Power serves as the target are more significant than those where Defense serves as the source. This indicates a stronger coupling relationships where Attack Power serves as the target are prominent, and those where Defense serves as the source are also prominent.
- Source/Target characteristics for individual attributes: Coupling relationships where Defense Growth $GDEF$ serves as the source exhibit the highest overall strength. Stamina is a "coupling island," exhibiting coupling only between stamina and stamina growth, with no cross-attribute coupling relationships.

Heatmaps convert numerical values into perceptible "coupling strengths" through intuitive color distributions. In particular, the fact that Stamina is a "coupling island" is especially evident in the figure. Furthermore, based on these overall conclusions, the design insights we derive are more holistic.

- Overall distribution characteristics: There are currently no coupling channels from "attribute values $\rightarrow$ growth values." This means players cannot sacrifice current attributes to gain long-term growth. Filling this gap could increase strategic depth during deckbuilding, such as by designing a card that "consumes Defense to boost Attack growth." Furthermore, there is no coupling between growth values. This implies that the growth of each attribute is independent, and there is no "trade-off" strategy for growth allocation.
- Distribution characteristics within blocks: In the AG sub-block, consider adding cards that boost Attack-HP growth to diversify the types of coupling within the AG block.
- Characteristics of single-attribute sources/targets: Adjustments involving defense growth $GDEF$ require particular caution, as higher coupling strength amplifies their impact. Adjustments to cards related to stamina can be more flexible and, when necessary, serve more sensitive attributes, such as attack and defense growth.

These conclusions, derived from the C_{AbsSum} matrix and its corresponding heatmap, provide a structural interpretation unique to commutator analysis, for which traditional analytical methods have no direct counterpart.

4.5.2 Coupling Frequency Heatmap

Previously, we plotted a coupling strength heatmap, which explained the overall strength of coupling between different parameters. However, the strength itself is not fixed; the effect of a pair of strongly coupled cards may appear more prominent on the heatmap than that of multiple pairs of weakly coupled cards. Therefore, in addition to strength, we also need a heatmap measuring coupling frequency to enable a more comprehensive analysis and comparison. By counting the positions and occurrence frequencies of all non-zero terms in the commutator matrix, we can plot the following coupling frequency diagram.

The coupling frequency plot is generally similar to the coupling strength plot, but some subtle differences are worth noting. We found that in the AG block, the relative relationship between $(ATK, GATK)$ and $(ATK, GDEF)$ is opposite to that shown in the coupling strength plot 4.2 at the same location. In the coupling frequency plot, $(ATK, GATK)$ is more prominent, whereas the opposite is true in the coupling strength plot. This indicates that the $(ATK, GATK)$ coupling exists between more cards but has lower strength. Conversely, $(ATK, GDEF)$ represents a rare yet highly influential coupling relationship. This difference reveals rich information related to card design.

The $(ATK, GATK)$ coupling is widely present across many card pairs, but the strength of each coupling is low. From a design perspective, this is a "foundational synergy." Players encounter the interaction between Attack and Attack Growth in various card combinations, but the impact of each interaction is relatively mild. In contrast, while the $(ATK, GDEF)$ coupling appears only in a small number of card pairs, its intensity is significant whenever it occurs. This is a "burst-style

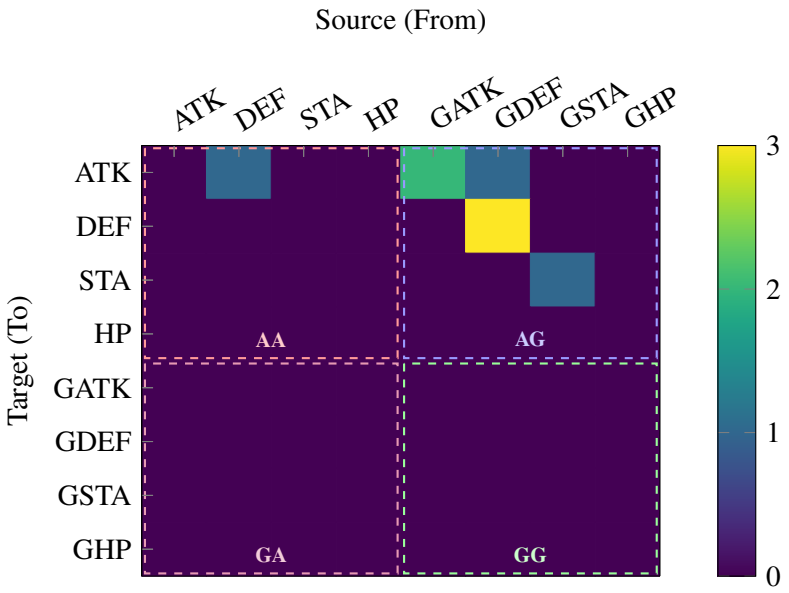


Figure 4.3. Heatmap of the frequency of non-zero terms among commutators

synergy." It is rarely triggered, but when it is, its impact on the game state far exceeds that of the basic synergy. Designers need to pay special attention to card combinations involving this channel: they may become the key to victory in player decks, or they may cause balance issues due to excessively high coupling intensity.

The two-dimensional analysis of frequency and intensity provides designers with a potential means of "coupling localization." High-intensity areas are balance hotspots requiring fine-tuning, while high-frequency areas represent routine interactions that form the foundation of the card pool's strategy. Conversely, areas with low intensity and low frequency (such as the Stamina-related dimension) represent relatively safe design spaces where higher-risk new mechanics can be introduced when necessary.

The coupling intensity distribution chart and the coupling frequency distribution chart provide a powerful visualization of the game's overall coupling structure using the commutator analysis method. They offer us a new perspective and analytical dimension for understanding the game's overall structure. At the same time, in the context of overall game analysis, these are merely among the most basic and intuitive methods. By summing the commutator matrices between a single card and all other cards, we can generate a heatmap for that individual card. Alternatively, we can calculate the norm of the commutators to measure their "magnitude," enabling further analysis and more. With the foundation of commutators, there is much more we can achieve—this is merely the starting point.

4.6 Summary of Case Study

Taking the auto-battler deck-building phase of Chaos Fighter as a case study, we have demonstrated a systematic application of the commutator analysis framework—from local to global, and from analysis to diagnosis.

Reviewing the entire analysis path: We first represent the effects of unique cards as block matrices, following the matrix element interpretation rules established in Chapter 3: rows represent targets, and columns represent sources. We first analyzed the relationships among three unique cards, and subsequently analyzed the impact of newly added cards on the existing system. In this process, pairwise commutator analysis distinguished three types of coupling within the current card system: attribute-internal transformation coupling (AA blocks), attribute-growth coupling (AG blocks), and fixed-value coupling (homogeneous columns). The location, strength, and di-

rection of each type of coupling were precisely quantified, and symbolic generalization revealed the sources of the coupling parameters.

Combinatorial commutator analysis further reveals deep-seated structures that traditional testing cannot access. By systematically comparing three external cards (M_2, M_3, M_4) with three combinations M_1M_2, M_1M_3, M_2M_3 , we discovered the existence of "internal order sensitivity within combinations" and its hierarchical differentiation: M_4 is completely insensitive to M_1M_2 because ($C_{4,12} = C_{4,21}$) ; M_4 is weakly sensitive to M_1M_3 (the difference between $C_{4,13}$ and $C_{4,31}$ is 0.02), ; and M_4 is significantly sensitive to M_2M_3. $C_{4,23}$ has one more coupling channel than $C_{4,32}$, and the strength of the other channel differs by 0.12. This level of detail is completely invisible in traditional testing: at most, testing can detect "different final outcomes," but it cannot distinguish between "the presence or absence of a channel" and "changes in strength," let alone trace these back to the exact product of three-way parameters.

Coupling strength heatmaps and coupling frequency heatmaps elevate commutator analysis from a local to a global perspective. The heatmaps reveal the structural characteristics of the current card pool: the AG block is a coupling hotspot, the GA and GG blocks are completely blank, GDEF is the most active source of coupling, and Stamina is a coupling island. The two-dimensional comparison of intensity and frequency further distinguishes between "basic synergies" (e.g., $(ATK, GATK)$, which are widespread but mild) and "burst synergies" (e.g., $(ATK, GDEF)$, which are rare but intense), providing precise prioritization guidance for designers in allocating balance testing resources.

These structural insights have no equivalent in traditional testing methods. Testing can reveal 'which card combinations are strong together,' but it cannot systematically demonstrate 'which attribute dimensions commonly exhibit coupling,' 'which dimensions are completely decoupled,' or 'which cards serve as coupling hubs.' Yet these are precisely the core outputs we obtain through the commutator analysis framework.

Furthermore, through further analysis of the combination commutators, we verified that the difference between $C_{4,23}$ and $C_{4,32}$ is exactly equal to the nested commutator $[M_4, [M_2, M_3]]$, which traces the source of the difference to the product of the attack gain ratio of Original Combustion, the defense conversion ratio of Swords Made from Armor, and the growth resolution multiplier of Stealing the Future. This equation links the three levels of commutators into a self-consistent analytical framework: pairwise coupling, combinatorial coupling, and nested coupling.

From a broader perspective, the analytical approach we present is automatable. The definition of the card matrix, the calculation of commutators, the generation of heatmaps, and the two-dimensional statistics of frequency and intensity can all be performed automatically by a program. As the card pool expands, designers do not need to manually review all commutator results one by one—heatmaps automatically highlight coupling hotspots and blind spots, combination commutator comparisons automatically flag combinations sensitive to internal order, and nested commutators automatically filter out second-order interactions requiring attention. This potential for automation makes commutator analysis suitable not only for manual analysis of small card pools but also for large-scale balance management in commercial-grade card games.

5. Discussion

5.1 Formalization and Mathematics

5.1.1 The Commutator as a Metric for Structural Coupling

Inspired by quantum mechanics, we developed a method for calculating and analyzing commutators applicable to games. When designing unique cards in our case study, we adopted the traditional designer's perspective—creating cards based on specific design goals. However, by computing commutators, we discovered a wealth of interactions between cards that had gone unnoticed during the design phase. As our analysis deepened, the insights revealed by the commutators became increasingly profound and nuanced. For a small card pool containing only four unique cards, this information was even overwhelming. All of this was accomplished without conducting any actual playtesting. Therefore, as a state-independent and directly computable measure of structural coupling, commutators offer extremely powerful insights into game structure.

5.1.2 From Analysis to Design

In the case study, the numerical differences between combination commutators prompted us to further explore the potential of symbolic matrices. Through the symbolic matrix, we were able not only to identify the sources of these differences but also to precisely control them. At the same time, the global structural perspective provided by the heatmap not only revealed information about the game's current coupling structure but also served to cross-validate with the designers' intuition. This contrast can significantly calibrate designers' understanding of the game's coupling layers. This demonstrates that commutator analysis is not merely an analytical tool but can also directly serve the design process.

5.1.3 Translation Process

One of the greatest challenges of this research lies in establishing a direct connection between complex mathematical concepts—such as matrix operations—and game design. The key lies in translating mathematical language into design terms. Without sufficient mathematical background, matrices alone are nearly impossible to interpret. Therefore, creating visual diagrams and reinterpreting mathematical concepts have become essential for effective communication. Translating the positions of matrix elements from row-column coordinates into "target-source" relationships effectively alleviates comprehension barriers caused by a lack of prior knowledge. For design practitioners, an intermediate context must be created to facilitate this conceptual bridge.

5.2 Game Structures and Applicable Scope

5.2.1 The Challenges of Tic-Tac-Toe

In the early stages, we used Tic-Tac-Toe as a starting point and attempted to model it. However, we soon discovered that while Tic-Tac-Toe appears simple, it is difficult to analyze using commutator analysis. To describe Tic-Tac-Toe, we attempted to encode its state as a 9+1-dimensional vector, with each dimension representing the state of a single grid cell. Furthermore, to represent all possible player moves, we required 18 10x10 matrices for all possible actions of two players. This

is a cumbersome model that bears no resemblance to the game's apparent simplicity. Additionally, after completing the model, analyzing and interpreting the results of the commutator calculations proved extremely difficult. In contrast, while Chaos Fighter features many potential, distinct cards and their corresponding effects, each card corresponds to only one matrix, and the dimension is limited to 8+1. Moreover, the commutator analysis is easy to interpret and holds great potential for design guidance. This requires us to reflect on the applicability of commutator analysis methods and the reasons for their suitability or unsuitability.

5.2.2 Logical Structures and Dynamical Structures

For a game, we can attempt to categorize its operational design as either a logical structure or a dynamical structure.

Logical structures are primarily based on state transitions and explicit rules. In a logical structure, operations are mainly defined by explicit logical relationships. For example, in Tic-Tac-Toe, "a space cannot already contain a piece," and in the 'Hostage' level of *Detroit: Become Human*, "picking up a gun indoors unlocks the 'Shoot' option." These are all actions and their constraints established by logical relationships.

Dynamical structures, on the other hand, are primarily based on numerical calculations and boundary conditions. Under a dynamical structure, actions manifest as the calculation and evolution of parameters. For example, in *The Elder Scrolls V*, the spell "Flames" continuously drains the caster's mana and reduces the health of affected units; in Horizon, operations such as accelerating, braking, and drifting are recreations of real-world physical processes.

In reality, most games are a hybrid of the two. Therefore, logical structures and dynamical structures are not opposing design approaches, but rather two distinct dimensions of design and implementation that are frequently used in combination. For example, in *Civilization VI*, the outcome of two units attacking each other, or the accumulation of productivity toward wonder construction, are closer to dynamical mechanisms. On the other hand, scoring during historical moments or determining victory conditions rely heavily on predefined logical conditions and branching paths established by the designer.

It must be emphasized that the fundamental distinction between logical and dynamical structures does not lie in the "continuous" versus "discrete" nature of time or space, but rather in how operations are defined and implemented within the system. High-frequency logical condition checks and jumps may feel continuous in terms of user experience, but they essentially still belong to logical structures. A continuously executed hitbox collision check and event trigger still constitutes a state transition driven by causal logic and/or conditional branching. Conversely, discrete, one-time numerical skills still fall within the scope of dynamical construction. In each of the cards analyzed in the previous case studies, the underlying mechanism is also numerical calculation and state evolution under boundary conditions.

5.2.3 Applicable Scope of Commutator Analysis

Once we have clarified the key distinction between logical and dynamical structures, the scope of application for commutator analysis becomes clearer. Commutator analysis is a method based on matrices, vectors, and state spaces, with its foundations rooted in numerical computation. Therefore, commutator analysis is primarily applicable to games dominated by dynamical structures and the dynamical components of hybrid games. For logical structures, however, commutator analysis incurs additional encoding and decoding costs. We must first encode logical states as vectors and operations as matrices, and then decode the computed commutator matrix elements. This makes commutator analysis not only more computationally intensive but also less applicable compared to traditional analytical methods at present. The distinction between logical and dynamical structures also explains why commutator analysis is difficult to apply to Tic-Tac-Toe, whereas it shows great potential for Chaos Fighter. This is because Tic-Tac-Toe is a purely logical game, whereas

the card-based mechanics in Chaos Fighter involve dynamical, numerically evolving operations. Therefore, commutator analysis should be applied to the dynamical components of the game and combined with other methods suitable for logical structures to form a comprehensive analysis of the game's logical and dynamical components.

Finally, it is necessary to emphasize that dynamical structures are a broader and more general concept than "matrixability." Other continuous or discrete numerical evolution rules, such as those relying on function evaluation or differential equations, also constitute dynamical structures. At the current stage, commutator analysis is primarily suited to the subclass of these that can be well-represented by linear matrices, but this does not constitute a limitation on the boundaries of dynamical structures.

5.3 Atomic White-Box Assumption and Games as Gray-Boxes

In our previous discussion, we implicitly assumed a certain understanding of games: that the commutator analysis method is based on a complete grasp of the game's fundamental components. We have explicitly termed this the "atomic white-box assumption," and we have shown that even if a game satisfies this assumption, there remains a vast black-box component. This is where emergent gameplay resides, and it is also the area where commutator analysis holds the greatest potential for application.

5.3.1 Atomic White-Box Assumption

In all our analyses, we consistently treat the game as an atomic white-box: there are no hidden dimensions or unknown effects regarding its fundamental components. In case studies, individual cards are the fundamental components, and all cards are complete and unambiguous. This assumption is reasonable and effective because its primary intended audience consists of designers and developers, both of whom have a complete understanding of the game's mechanics, rules, and other fundamental structures. However, for external observers—such as players or analysts—the game is no longer an atomic white-box. Observers must infer and construct approximate models, and the validity of commutator analysis is thus influenced by the precision of these models. Consequently, the reliability of commutator analysis depends on a complete understanding of the game's mechanics. An incomplete understanding will affect the validity of analyses—such as those of coupling channels—to varying degrees.

5.3.2 Games as Gray-Boxes

It should be noted that atomic white-box does not imply complete mastery or prior understanding of the game system. We need to make a more nuanced distinction:

Atomic White-Box: The effects of individual actions are typically known. The logical relationships between individual rules are clear when they are described unambiguously. This is designed and defined by the designer themselves, and is an absolute white-box scenario for the designer.

Coupled Black-Box: When multiple actions are combined, or when different rules interact, the resulting effects are often a black-box to the designer. Specifically:

Logical black-boxes primarily stem from the explosion of state combinations, the mutual influence of multiple constraints, or complex truth value transitions. For example, two plot events that should be mutually exclusive might occur simultaneously. Dynamical black-boxes arise from parameter resonance or boundary breaches. For instance, a loop involving three operations can produce a net effect with no cost or boundary constraints.

The vast majority of real-world games are gray-boxes: their atomic structures are clearly white-boxes, while their coupled structures are complex and massive black-boxes. Unexpected phenomena encountered in game testing, such as "operation deadlocks" or "overpowered combinations,"

are essentially the result of exploring the black-box components. Furthermore, absolutely white-box games do indeed exist. Any game that can be fully described by a finite state machine is an absolutely white-box game, because all possible states and state transitions are completely explicit. However, in practice, due to the sheer number of states, even a mathematically white-box game cannot be fully grasped by humans. In terms of cognition and computation, games such as Go should still be regarded as gray-boxes.

5.4 Commutator and Emergence

5.4.1 Emergence and the Black-Box

There has been extensive discussion and analysis regarding emergence in games. In *Rules of Play*, Tekinbas and Zimmerman (2003) explore the characteristics of games as emergent systems. They explicitly state that the key to emergence in games is that "complex possibilities are the result of a simple set of rules". Here, "a simple set of rules" corresponds to the atomic white-box, while "complex possibilities" represent the coupled black-box component. Adams and Dormans (2012) offer a similar discussion in *Game Mechanics: Advanced Game Design*. These studies can all be categorized as descriptions and analyses of the black-box aspects of games.

5.4.2 The Relationship Between Commutator and Emergence in Games

Traditionally, the discovery of emergence has relied almost entirely on testing or feedback from the player community. This is essentially a form of random trial and error or sampling. However, guided by experience, purposeful testing offers greater focus and efficiency compared to completely random methods. Commutator analysis can, in fact, be viewed as a method for the structural analysis and exploration of the dynamical black-box. Holland (2000) notes that "emergence is above all a product of coupled, context-dependent interactions". And commutator analysis is precisely an analysis of the coupling relationships between operations within a game. Therefore, non-zero elements in the commutator matrix actually mark a potential for emergence.

By analyzing and calculating commutators, we can assess potential emergent phenomena as early as the design phase. For example, in the discussion of the distinction between logical inversion and mathematical inversion, the "net extraction effect" of logical inversion is a macroscopic manifestation of the distribution of coupling channels in the results of commutator calculations. Similarly, the conversion between attack power and defense growth in Chaos Fighter represents an emergent coupling relationship that does not exist in atomic operations. This analysis is neutral and requires judgment based on the designer's objectives. Designers can either retain beneficial emergent phenomena or specifically eliminate and prevent those that do not align with their design goals.

5.4.3 Designed Emergence?

A question arises: Can emergence be designed? In fact, Adams and Dormans (2012) have raised a similar question. Their answer was to study specific structures within game mechanics and utilize them to create gameplay. However, the commutator analysis offers another possibility. When we wish to induce or suppress a certain type of dynamical emergence, we can adjust the matrix representation of operations to directionally create or block coupling channels within the commutator matrix. While this cannot fully determine the final form of emergence, it provides designers with a lever to actively influence the likelihood of emergence during the design phase. This can help us shift our approach to emergence from passive discovery to partial active prediction and management—and even design.

5.4.4 The Role and Position of Commutator Analysis

Emergent phenomena in games are a highly complex and multidimensional issue. Aspects such as the "logical black-box" or the emotional experiences derived from emergent gameplay fall outside the scope of the commutator matrix analysis. In summary, we can position the commutator matrix analysis as a tool capable of conducting a structured scan of dynamical emergence from the designer's perspective during the design phase. This holds the promise of transforming emergence management from an empirical craft reliant on extensive testing into a discipline with a computable, systematic engineering foundation.

5.5 Limitations and Boundaries

As stated at the end of Chapter 3, the current commutator analysis framework is limited to linear, state-independent operation representations. More common state-dependent operations in games—such as conditional effects like "Attack Power +3, or +6 if current Attack Power is greater than 80"—are not yet covered by this framework.

In our case study, we validated the feasibility of this method within a small card pool. However, as the number of cards increases, the number of commutators and nested commutators will grow exponentially. This poses challenges to both the difficulty and effectiveness of the analysis. In large-scale scenarios, the feasibility of the analysis and the specific methods employed still require practical validation.

Furthermore, commutator analysis focuses on the objective structure of the game, whereas the complete gaming experience resides in the player's gameplay process. Therefore, this analytical method cannot replace more multidimensional game testing, particularly in terms of gameplay experience and emotional feedback. Commutator analysis should currently be positioned as an analytical tool offering a specific perspective, rather than a comprehensive takeover of game testing.

6. Conclusions and Future Work

6.1 Addressing the Research Questions

Our initial research questions were:

Can commutative relations and commutators serve as a direct, structural, quantitative measure of game action coupling, revealing structural information that is difficult or impossible to obtain through existing testing and simulation methods? If so, how can they be applied in practical game design?

Our answer is affirmative. The structural information provided by combination commutators, nested commutators, and coupling heatmaps derived from commutators, cannot be directly obtained through existing testing and simulation methods. In our case studies, we have provided specific design insights based on commutator analysis. Therefore, in practical game design, commutators are expected to serve as an auxiliary tool, revealing objective structural information about coupling and thus becoming an important reference for designers during the design process.

6.2 Theoretical Contributions

Based on matrix algebraic operations and commutative relations, we have established an algebraic commutator tailored to the context of game design and analysis. In this context, the algebraic commutator is interpreted as a tool for measuring coupling. We have provided rules for interpreting game operation matrix elements and commutator matrix elements within the game context, clarifying the meaning of row and column coordinates as "target-source" pairs. Furthermore, through the identity transformation $AB = \frac{1}{2}\{A,B\} + \frac{1}{2}[A,B]$, we extend the comparison of coupling between different sequences to a measure of coupling effects within a single combination operation: the deviation of the operation combination relative to the average effect. We also distinguish between logical inverses and mathematical inverses—concepts often conflated in design practice—through the calculation of commutators, demonstrating the potential of commutators for conceptual clarification.

6.3 Contributions of the Case Study

In the case analysis of Chaos Fighter, by representing matrices through partitioning, we further clarify the classification of coupling types among different parameters. Through the calculation and analysis of combinatorial commutators, we have revealed a level of "internal order sensitivity within combinations" that is difficult to access through traditional testing. By plotting coupling strength and coupling frequency heatmaps based on case-specific cards, we have successfully identified coupling hotspots, coupling blind spots, and key coupling parameters within the current card pool. The layer-by-layer analytical approach used in the case study is not only practical for small card pools, but also characterizes the game's overall coupling structure across multiple levels.

6.4 Outlook for Nonlinear Scenarios

The analysis of nonlinear operations is a major future research direction. One possible approach is to use a parameterized matrix family $M(\lambda)$(Horn & Johnson, 2012). A parameterized matrix family represents a class of matrices; each time an operation is applied, a specific matrix is

selected from the family based on the current parameter λ, while the matrix itself remains state-independent. For conditional evaluation operations, one can also consider a convex combination of matrices:

$$M(b) = bA + (1-b)I$$

When the Boolean value $b = 0$, $M(0) = I$ is the identity matrix, indicating no effect; when $b = 1$, $M(1) = A$, which triggers the target effect. The above approach is currently still at the conjectural stage and requires further verification and refinement.

6.5 More General Commutators

Algebraic commutators are merely one form within the family of commutators; others include group commutators and Lie brackets. Group commutators measure coupling relationships via the cycle $[A,B]_g = ABA^{-1}B^{-1}$(JS, 2022), but they require the existence of inverse operations—which is precisely one of the reasons we distinguish between logical inverse and mathematical inverses. Lie brackets, on the other hand, are measures of coupling for instantaneous effects in continuous settings(Hall, 2013), and are of critical importance for the transition from discrete game analysis to continuous game analysis. Algebraic commutators, group commutators, and Lie brackets are all, in essence, measures of coupling; the distinction lies in their different scopes of application. Therefore, the other two types of commutators, which have not yet been fully explored in this study, hold significant potential for future research.

Bibliography

Aarseth, E. J. (1997). *Cybertext: Perspectives on ergodic literature*. JHU Press.

Adams, E., & Dormans, J. (2012). *Game mechanics: Advanced game design*. New Riders.

Baier, C., & Katoen, J.-P. (2008). *Principles of model checking*. MIT press.

Berlekamp, E. R., Conway, J. H., & Guy, R. K. (2004). *Winning ways for your mathematical plays, volume 4*. AK Peters/CRC Press.

Bond, J. G. (2014). *Introduction to game design, prototyping, and development: From concept to playable game with unity and c*. addison-wesley professional.

Brogan, W. L. (1985). *Modern control theory*. Pearson education india.

Cover, T. M. (1999). *Elements of information theory*. John Wiley & Sons.

Dormans, J. (2012). Engineering emergence: Applied theory for game design.

Hall, B. C. (2013). Lie groups, lie algebras, and representations. In *Quantum theory for mathematicians* (pp. 333–366). Springer.

Holland, J. H. (2000). *Emergence: From chaos to order*. OUP Oxford.

Horn, R. A., & Johnson, C. R. (2012). *Matrix analysis*. Cambridge university press.

Hunicke, R., LeBlanc, M., Zubek, R., et al. (2004). Mda: A formal approach to game design and game research. *Proceedings of the AAAI Workshop on Challenges in Game AI*, *4*(1), 1722.

JS, R. D. (2022). *Abstract algebra* (Vol. 1). De Gruyter.

Juul, J. (2011). *Half-real: Video games between real rules and fictional worlds*. MIT press.

Khalil, H. K., & Grizzle, J. W. (2002). *Nonlinear systems* (Vol. 3). Prentice hall Upper Saddle River, NJ.

Nelson, M. J., & Mateas, M. (2007). Towards automated game design. *Congress of the Italian Association for Artificial Intelligence*, 626–637.

Newman, M. (2018). *Networks*. Oxford university press.

Russell, S. J. (2010). *Artificial intelligence a modern approach*. Pearson Education, Inc.

Skogestad, S., & Postlethwaite, I. (2005). *Multivariable feedback control: Analysis and design*. john Wiley & sons.

Tekinbas, K. S., & Zimmerman, E. (2003). *Rules of play: Game design fundamentals*. MIT press.

Von Neumann, J. (2018). *Mathematical foundations of quantum mechanics: New edition*. Princeton university press.

Appendix A.
Value of Commutators in Case Study

Single - Single Commutators:

1. $C_{1,2}$

$$C_{1,2} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -4 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2. $C_{1,3}$

$$C_{1,3} = \begin{bmatrix} 0 & 0 & 0 & 0 & -0.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.4 & 0 & -10 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3. $C_{1,4}$

$$C_{1,4} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -0.5 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

4. $C_{2,3}$

$$C_{2,3} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0.6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

5. $C_{2,4}$

$$C_{2,4} = \begin{bmatrix} 0 & -0.12 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

6. $C_{3,4}$

$$C_{3,4} = \begin{bmatrix} 0 & 0 & 0 & 0 & -0.4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Single - Combined Commutators:

1. $C_{4,12}$

$$C_{4,12} = \begin{bmatrix} 0 & 0.114 & 0 & 0 & 0 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2. $C_{4,21}$

$$C_{4,21} = \begin{bmatrix} 0 & 0.114 & 0 & 0 & 0 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3. $C_{4,13}$

$$C_{4,13} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0.38 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & -0.38 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

4. $C_{4,31}$

$$C_{4,31} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0.4 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & -0.4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

5. $C_{4,23}$

$$C_{4,23} = \begin{bmatrix} 0 & 0.12 & 0 & 0 & 0.4 & 0.12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.28 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

6. $C_{4,32}$

$$C_{4,32} = \begin{bmatrix} 0 & 0.12 & 0 & 0 & 0.4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Appendix B.
Mathematica Source Code

Listing 6.1. Coupling Strength Calculation

```
(* 1. Define the initial state vector and the 9x9 homogeneous
   transformation matrix *)

s = {ATK, DEF, STA, HP, GATK, GDEF, GSTA, GHP, 1};

M1 = {

{0.95, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.95, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 1.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, -10},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 5},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M2 = {
{1.00, 0.30, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.70, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0.00, -20},
{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M3 = {
{1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0.00, 0.00, 0},
{0.00, 1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0.00, 0},
```

```
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0},
{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M4 = {
{1.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 10},
{0.00, 0.80, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 0.00, 1.10, 0.00, 0.00, 0.00, 0.00, 50},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, -10},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}

};

(* 2. Define a ternary commutator [Mk, Mi.Mj] = Mk.(Mi.Mj) - (Mi.Mj).Mk *)

numCommutator[A_, B_] := A . B - B . A;

(* 3. Iterate and collect all ternary commutator matrices that satisfy the conditions (24 combinations in total) *)

allCommutatorMatrices = Flatten[
Table[
If[k != i && k != j,

numCommutator[Symbol["M" <> ToString[k]], Symbol["M" <> ToString[i]] . Symbol["M" <> ToString[j]]],

Nothing
],

{k, 1, 4}, {i, 1, 4}, {j, 1, 4}

],

2
];

```

```
(* 4. Take the absolute value of all matrices and sum them by
   position *)

totalAbsoluteMatrix = Total[Abs[allCommutatorMatrices]];

(* 5. Format the output *)

Print["The sum of the absolute values of all commutator submatrices
   :"];

Print[MatrixForm[totalAbsoluteMatrix]];
```

Listing 6.2. Coupling Frequency Calculation

```
(* 1. Define the initial state vector and the 9x9 homogeneous
   transformation matrix *)

s = {ATK, DEF, STA, HP, GATK, GDEF, GSTA, GHP, 1};

M1 = {

{0.95, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.95, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 1.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, -10},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 5},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M2 = {
{1.00, 0.30, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.70, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0.00, -20},
{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M3 = {
```

```
{1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0.00, 0.00, 0},
{0.00, 1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0.00, 0},
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 2.00, 0.00, 0},
{0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, -3},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}
};

M4 = {
{1.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 10},
{0.00, 0.80, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0},
{0.00, 0.00, 0.00, 1.10, 0.00, 0.00, 0.00, 0.00, 50},

{0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, 0.00, 0},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1.00, -10},

{0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 1}

};

matrices = {M1, M2, M3, M4};

(* 2. Calculate the commutator submatrices of pairwise combinations
    [A, B] = A.B - B.A (6 combinations in total) *)

commutators = Table[pair[[1]] . pair[[2]] - pair[[2]] . pair[[1]],
   {pair, Subsets[matrices, {2}]}];

(* 3. Define a counting function: count the positions of non-zero
   elements on the off-diagonal *)

countNonDiagonalNonZero[matrix_] := Table[

If[i != j && matrix[[i, j]] != 0, 1, 0], {i, 1, 9}, {j, 1, 9}

];

(* 4. Summarize the counts of off-diagonal non-zero elements in all
    commutators *)

finalCountMatrix = Total[Map[countNonDiagonalNonZero, commutators
   ]];

```

```
96 (* 5. Format the output *)
97
98 Print["Statistical matrix of off-diagonal non-zero items:"];
99
100 Print[MatrixForm[finalCountMatrix]];
```